



Tema 18.1: Introducción a los LLMs

Contenidos

- Introducción al NLP
- RAG (Retrieval Augmented Generation)
- Transformers
- LLMs (Large Language Models)
- Como se entrena un LLM
- ¿Cuánto cuesta entrenar un modelo?
- Uso de un LLM
- Finetuning
- Razonamiento en Inferencia
- Explotación de modelos con Hugging Face
- Ejercicios Opcionales



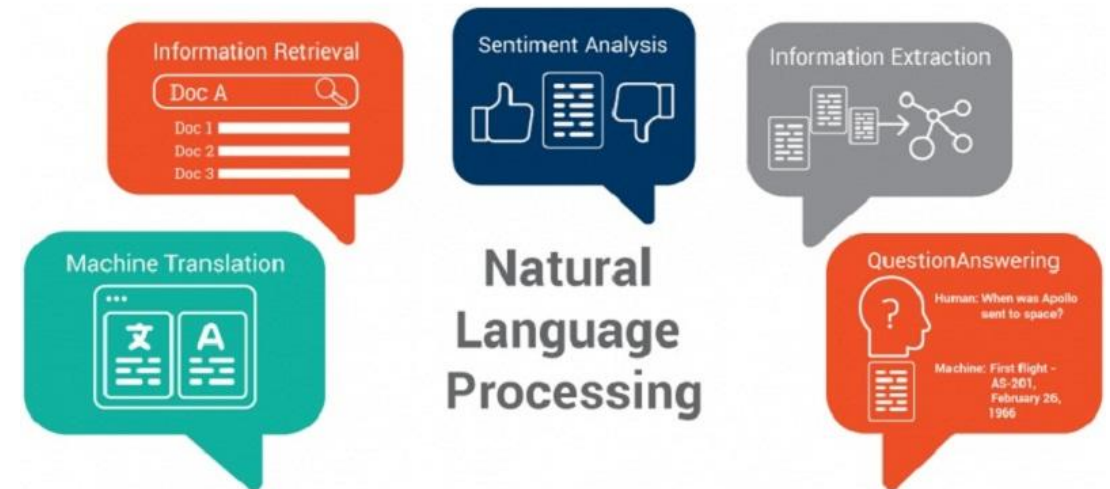
Introducción al NLP

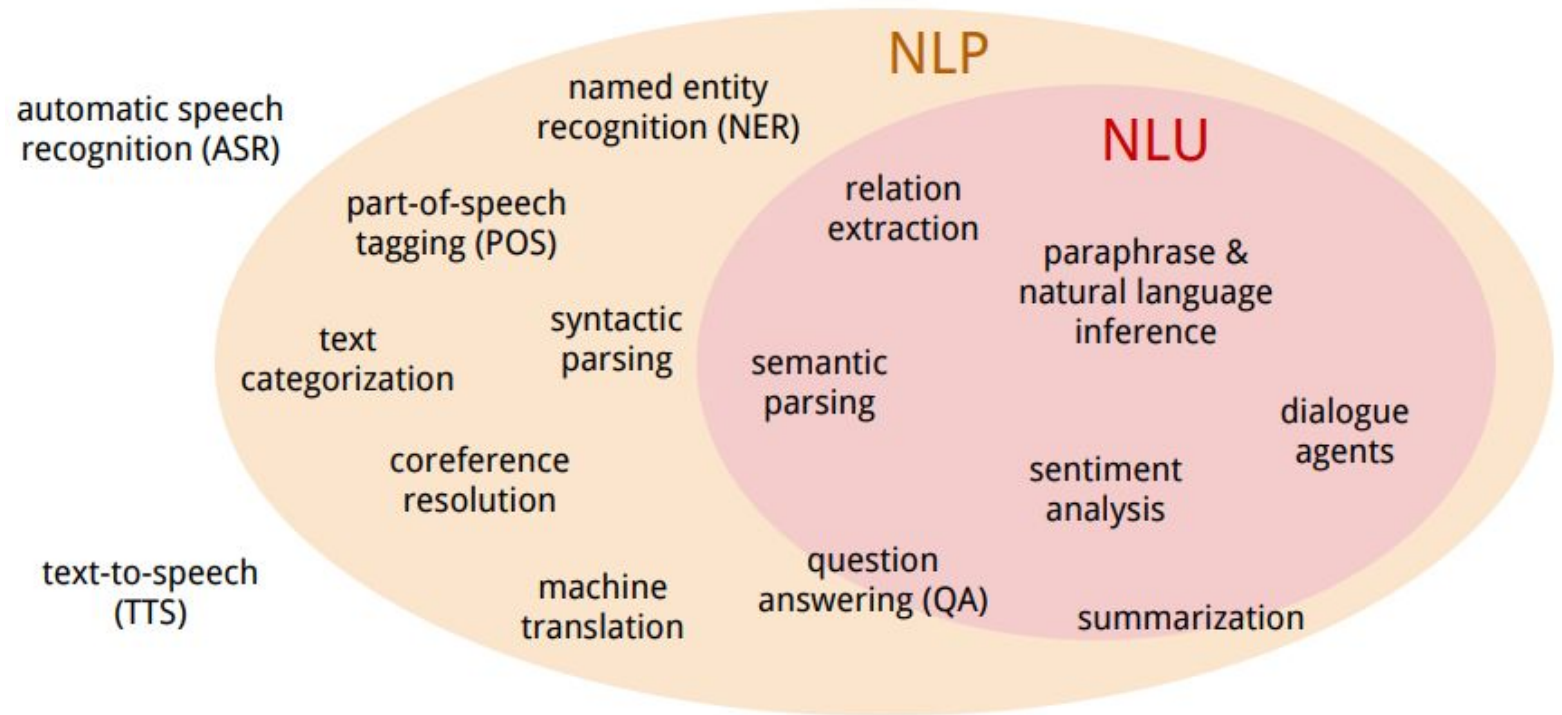
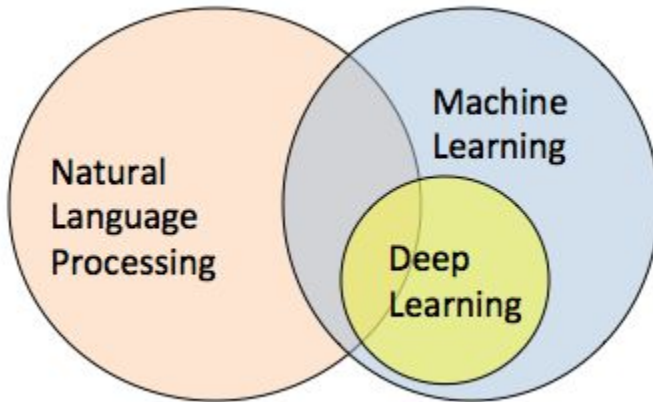


- Un lenguaje es un sistema potencialmente infinito de oraciones y expresiones construidas mediante reglas gramaticales, fonéticas y de significado que rigen la comunicación dentro de una comunidad.
- Nos encontramos con 3 tipos de lenguajes:
 - **Lenguaje Natural:** Son lenguas o idiomas que se desarrollan de forma espontánea a lo largo del tiempo en una comunidad de hablantes, utilizados para la comunicación diaria. Ejemplos: inglés, español, francés, italiano, etc.
 - **Lenguaje Formal:** Son lenguajes diseñados para un propósito específico, definidos de manera precisa y sin ambigüedades. Se utilizan en matemáticas, lógica, música, y programación, entre otros. Ejemplos: lenguajes matemáticos, lenguajes de programación como C, Java, Python, R, Scala, etc.
 - **Lenguaje Artificial:** Son lenguajes creados deliberadamente antes de ser usados por sus hablantes. No se desarrollan de manera natural, sino que son diseñados con un propósito específico, a menudo para obras de ficción o experimentos lingüísticos. Ejemplo: Klingon, Esperanto.

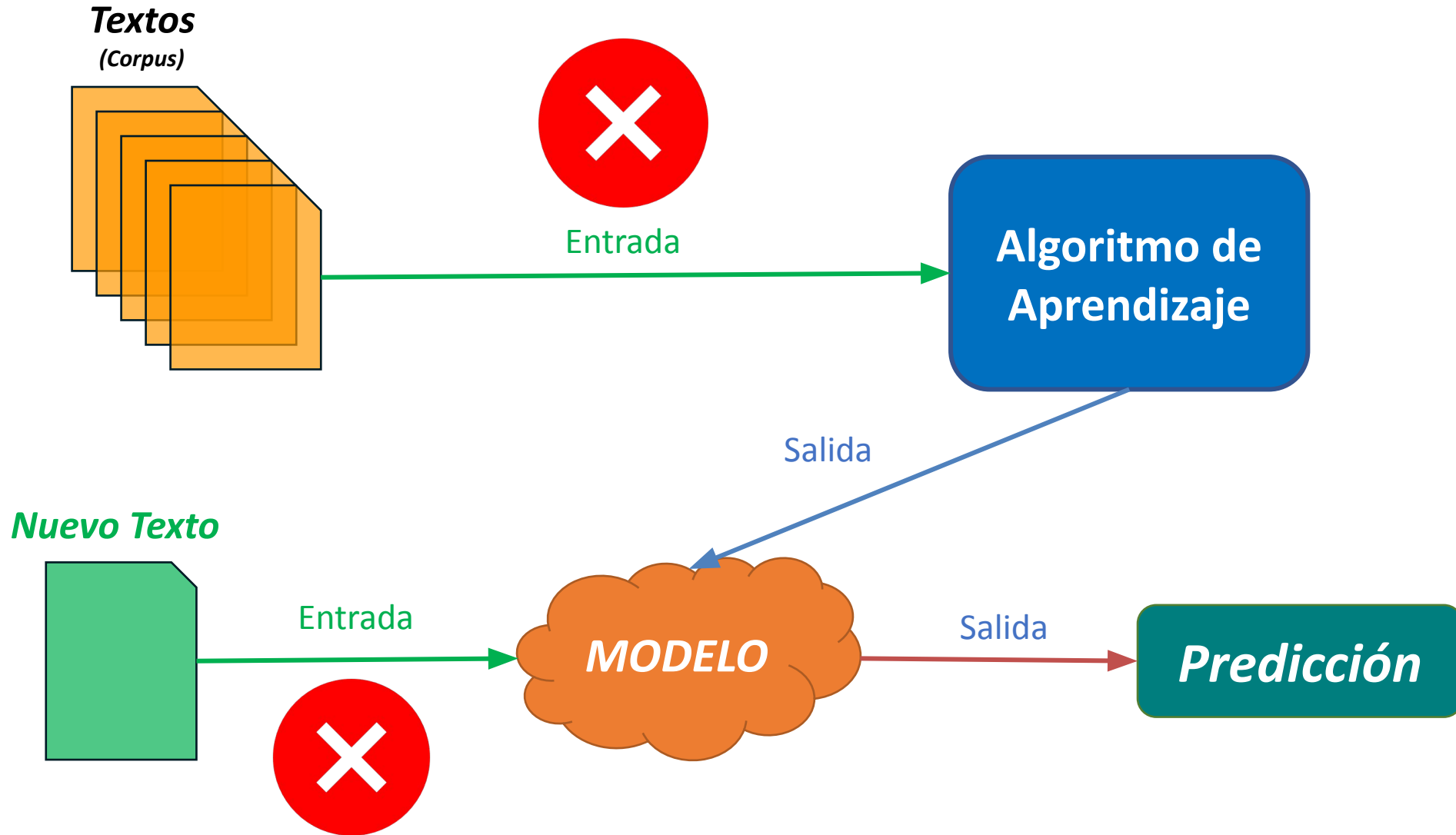


- El **Procesamiento del Lenguaje Natural** (NLP) es un campo que combina la **Informática**, la **Inteligencia Artificial** y la **Lingüística**; que tiene como objetivo, tratar la interacción entre los lenguajes humanos (lenguajes naturales) y los dispositivos informáticos.
- El NLP abarca los siguientes campos:
 - Recuperación de información.
 - Extracción y categorización de información.
 - Análisis automático de texto subjetivo.
 - Traducción automática.
 - Generación del lenguaje.
 - Questions & Answering (Chatbots).

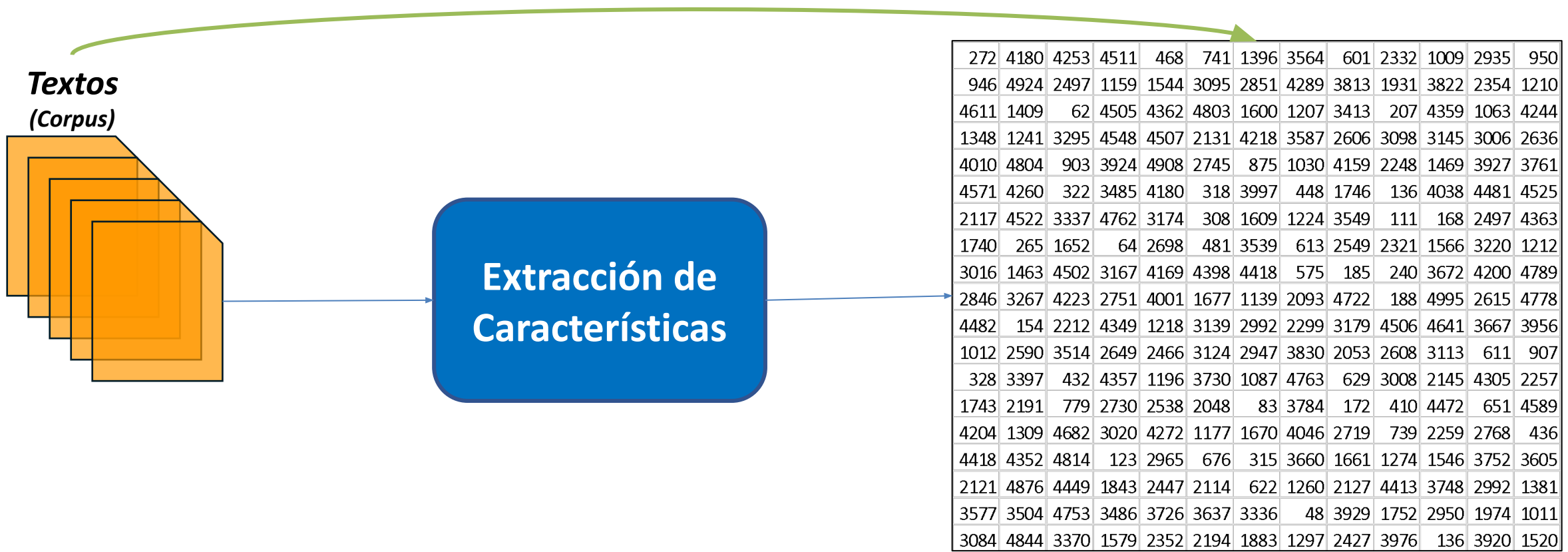




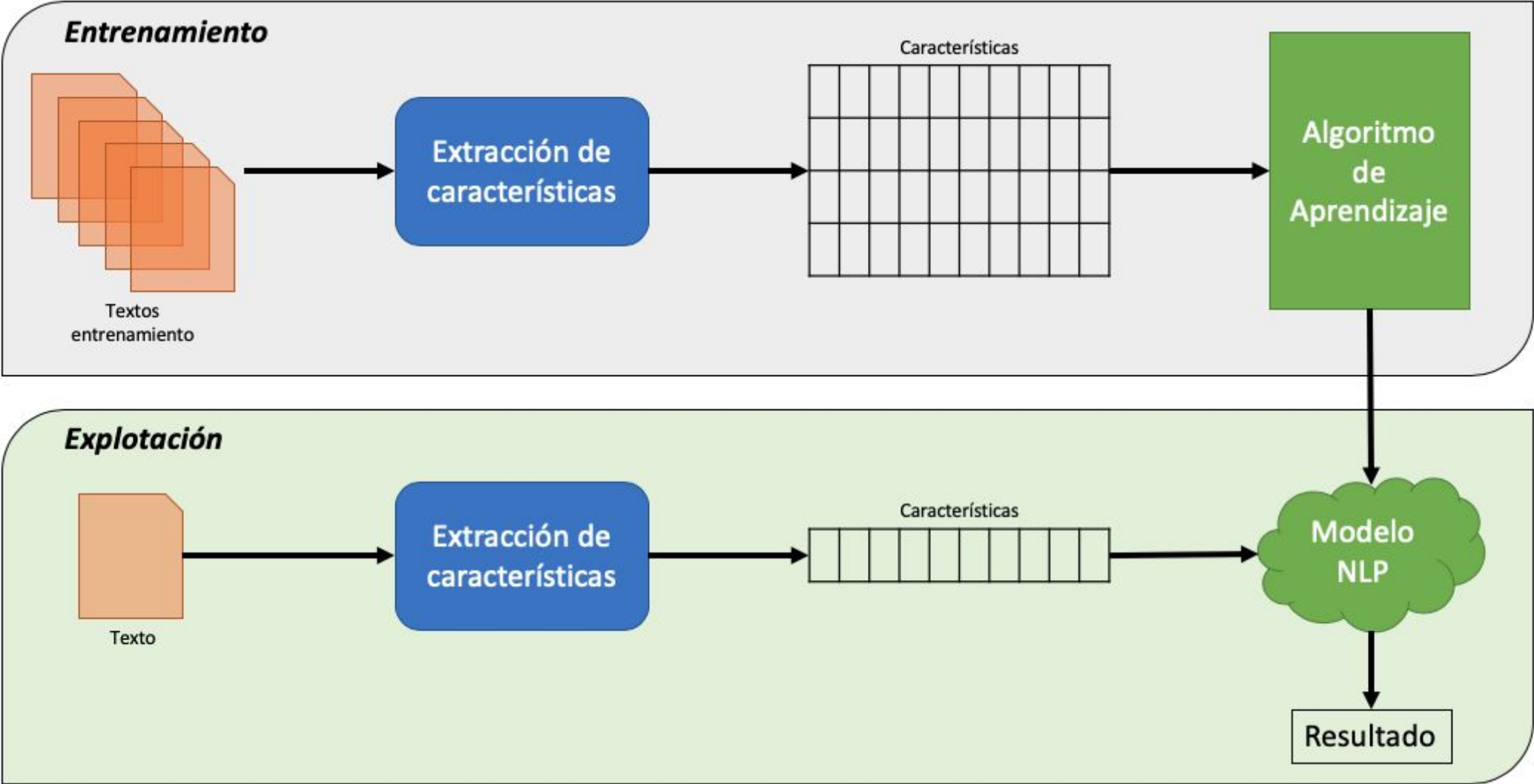
Los algoritmos de aprendizaje no entienden de palabras



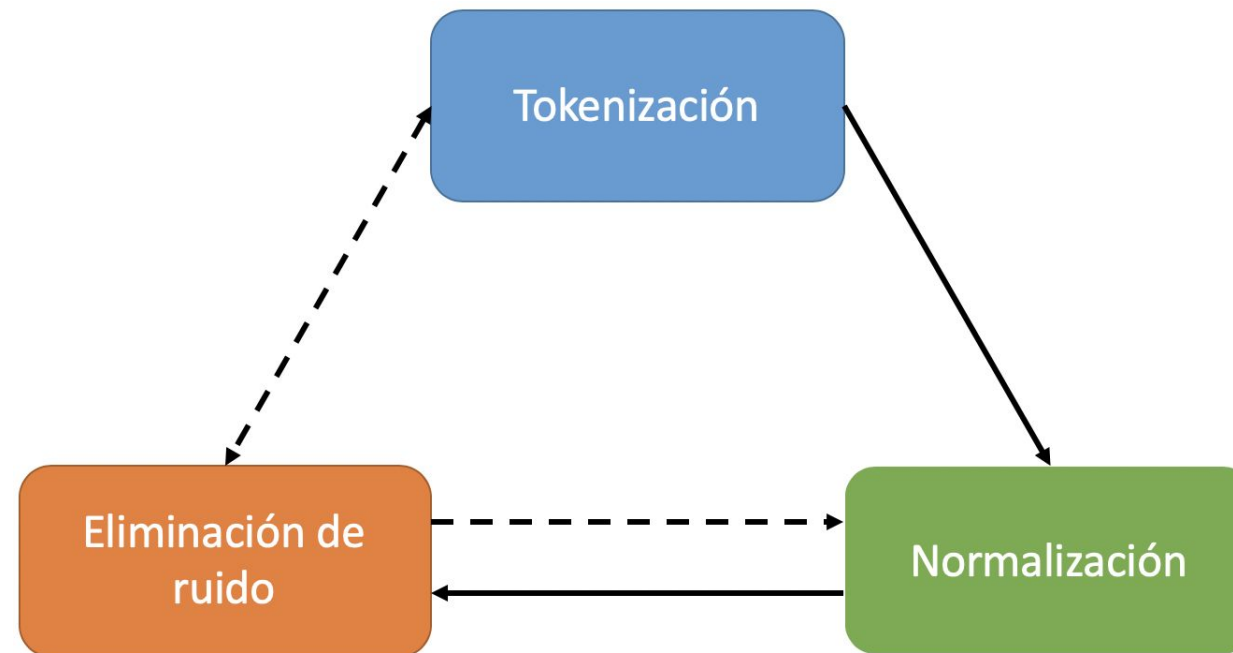
Los algoritmos de aprendizaje no entienden de palabras, entienden de números



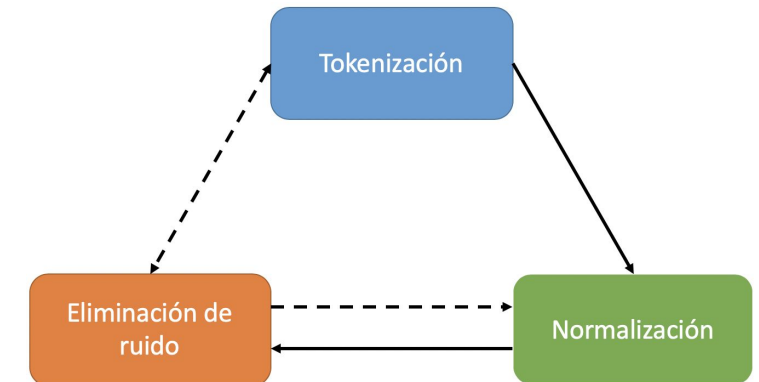
Los algoritmos de aprendizaje no entienden de palabras



- Antes de aplicar cualquier algoritmo de aprendizaje automático, es necesario realizar un preprocesamiento para limpiar, normalizar y estructurar los datos. El framework propuesto es:



1. **Eliminación de ruido:** Eliminar símbolos o caracteres irrelevantes, como etiquetas HTML, XML o JSON.
1. **Tokenización:** Dividir el texto en unidades más pequeñas o tokens, como palabras o subpalabras.
1. **Normalización:** Unificar el formato del texto:
 - Convertir todo a mayúsculas o minúsculas.
 - Eliminar signos de puntuación.
 - Convertir números a palabras si es necesario.
 - Eliminar “stop-words” (palabra con poco valor ‘el’, ‘y’, ‘de’...)



Text Data Hierarchy



Corpora



Corpus



Document



Token



1. Segmentación: Dividir el texto en unidades mayores, como oraciones o párrafos.
2. Tokenización: Dividir cadenas de texto en unidades más pequeñas llamadas tokens.

a. Tokenización Tradicional (Palabra por palabra)

- i. División de texto en unidades lingüísticas básicas, generalmente palabras.
- ii. Simple y directa
- iii. Basada en separadores (espacios, puntuación)
- iv. Cada palabra es un token

b. Tokenización Avanzada (Subword Tokenization)

- i. División de texto en unidades subléxicas que pueden ser más pequeñas que las palabras.
- ii. Utilizada en modelos modernos como GPT
- iii. Más eficiente en el manejo del vocabulario
- iv. Puede dividir palabras en componentes significativos



Texto

Un radar multa a Mariano Rajoy por caminar demasiado rápido.

tokenización palabra por palabra

Texto Tokenizado

['Un', 'radar', 'multa', 'a', 'Mariano', 'Rajoy', 'por', 'caminar', 'demasiado', 'rápido']



The Tokenizer Playground

Experiment with different tokenizers (running locally in your browser).

gpt-4 / gpt-3.5-turbo / text-embedding-ada-002 ▼

Un radar multa a Mariano Rajoy por caminar demasiado rápido.

TOKENS	CHARACTERS
16	61

Un radar multa a Mariano Rajoy por caminar demasiado rápido.

☒ Text ☐ Token IDs ☐ Hide

Fuente: <https://huggingface.co/spaces/Xenova/the-tokenizer-playground>



- La **lematización** es un proceso lingüístico que sustituye una palabra con forma flexionada (plurales, femeninos, verbos conjugados, etc.) por su lema; es decir, por una palabra válida en el idioma.

Texto

Unos radares multan a Mariano Rajoy por ir caminando demasiados rápidos.

Palabra	Lema
Unos	uno
radares	radar
multan	multar
a	a
Mariano	Mariano
Rajoy	Rajoy
por	por
ir	ir
caminando	caminar
demasiados	demasiado
rápidos	rápido



- Las **Stop Words** son palabras que (en teoría) no aportan nada de significado a la frase como, por ejemplo, determinantes, preposiciones, etc.

Palabras que suelen considerarse Stop Words (varía según la tarea y objetivos del modelo)

['había', 'ustedes', 'siempre', 'deprisa', 'día', 'mío', 'nadie', 'tambien', 'añadió', 'llevar', 'haber', 'como', 'contra', 'emplear', 'voy', 'ese', 'buenos', 'otras', 'próximos', 'realizar', 'mencionó', 'aquel', 'cierta', 'ella', 'apenas', 'vais', 'tus', 'conseguimos', 'tuyas', 'usamos', 'intentamos', 'ellas', 'dijeron', 'próximo', 'míos', 'quedó', 'agregó', 'ti', 'respecto', 'dejó', 'hace', 'se', 'éste', 'haciendo', 'ni', 'entre', 'esas', 'trabajais', 'da', 'poca', 'informo', 'podriais', 'dijo', 'hoy', 'cualquier', 'fin', 'nosotros', 'última', 'nuevo', 'esto', 'esos', 'nada', 'propio', 'vuestros', 'hecho', 'sera', 'ocho', 'conmigo', 'aquéllos', 'segunda', 'verdad', 'sean', 'tercera', 'general', 'conocer', 'todavía', 'nuevas', 'mia', 'ha', 'misma', 'ello', 'sé', 'porque', 'aseguró', 'cómo', 'este', 'era', 'propios', 'parte', 'año', 'sois', 'ser', 'están', 'trata', 'donde', 'señaló', 'adelante', 'tanto', 'mucho', 'quienes', 'deben', 'dice', 'cual', 'diferentes', 'lado', 'buena', 'esta', 'ningunas', 'hasta', 'soy', 'buen', 'verdadero', 'sí', 'otro', 'aquellas', 'sabeis', 'días', 'tener', 'después', 'dicho', 'dos', 'excepto', 'los', 'aun', 'cuáles', 'entonces', 'trabajamos', 'cierto', 'arribaabajo', 'intentais', 'tuya', 'parece', 'fuimos', 'las', 'través', 'intento', 'manifestó', 'unos', 'puedo', 'suyas', 'siendo', 'empleo', 'vamos', 'manera', 'que', 'por', 'algunos', 'cuenta', 'ahí', 'ejemplo', 'pesar', 'tan', 'mediante', 'vosotros', 'verdadera', 'eso', 'cuántas', 'dar', 'ningún', 'vuestra', 'nueva', 'informó', 'estamos', 'buenas', 'espacio', 'tenemos', 'pronto', 'propias', 'hacen', 'dieron', 'sí', 'mayor', 'del', 'repente', 'existe', 'todo', 'sigue', 'nuestro', 'estais', 'eramos', 'último', 'eras', 'pero', 'algunas', 'consigo', 'todavía', 'cuantas', 'fuera', 'quizas', 'fue', 'primero', 'el', 'creo', 'lleva', 'ayer', 'hablan', 'incluso', 'podemos', 'usted', 'vosotras', 'siete', 'sólo', 'lo', 'suya', 'aquella', 'mis', 'ex', 'cuatro', 'intentar', 'quiénes', 'muchos', 'comentó', 'atrás', 'empleamos', 'bajo', 'cuánto', 'durante', 'hizo', 'detrás', 'mas', 'ya', 'dado', 'estuvo', 'enfrente', 'para', 'haceis', 'han', 'podrá', 'aquéllas', 'pasado', 'llegó', 'mismos', 'antes', 'mucho', 'ningunos', 'tengo', 'esas', 'segundo', 'cuando', 'habrá', 'mi', 'usas', 'van', 'ahora', 'solo', 'ir', 'había', 'seis', 'cuándo', 'hicieron', 'tendrá', 'ciertos', 'muy', 'toda', 'tuyo', 'realizó', 'todas', 'al', 'hubo', 'podría', 'tiempo', 'consigue', 'sino', 'teneis', 'les', 'supuesto', 'mientras', 'algún', 'así', 'gueno', 'tenía', 'uno', 'estaba', 'tarde', 'podrias', 'valor', 'solamente', 'él', 'encima', 'nunca', 'país', 'principalmente', 'nuestras', 'eres', 'hemos', 'claro', 'cerca', 'no', 'igual', 'lejos', 'momento', 'días', 'hacia', 'quiere', 'son', 'habla', 'menos', 'pocos', 'te', 'nosotras', 'junto', 'cuál', 'intenta', 'saber', 'quizás', 'somos', 'posible', 'cuanto', 'luego', 'mismas', 'ellos', 'nuevos', 'cinco', 'usais', 'quizá', 'intentan', 'mismo', 'diferente', 'debe', 'usar', 'ambos', 'fui', 'horas', 'tiene', 'primer', 'está', 'tampoco', 'medio', 'allí', 'trabajar', 'consideró', 'propia', 'dio', 'debido', 'vaya', 'pueda', 'demasiado', 'pocas', 'estos', 'podrán', 'estas', 'varias', 'va', 'sus', 'aquél', 'será', 'demás', 'estados', 'primeros', 'sabemos', 'consiguen', 'hay', 'alguna', 'podrian', 'ninguno', 'vuestras', 'quien', 'país', 'esos', 'hacer', 'trabajan', 'ver', 'hago', 'proximo', 'aquí', 'queremos', 'explicó', 'debajo', 'sabes', 'ante', 'qué', 'trabajas', 'esa', 'estaban', 'partir', 'estará', 'siguiente', 'tuyos', 'sola', 'tres', 'largo', 'algo', 'le', 'cuánta', 'nuestros', 'primera', 'trabaja', 'según', 'de', 'considera', 'sería', 'uso', 'dentro', 'después', 'eran', 'pasada', 'muchas', 'con', 'menudo', 'enseguida', 'casi', 'conseguir', 'poder', 'grandes', 'contigo', 'antano', 'varios', 'tuvo', 'aquella', 'arriba', 'éstos', 'nos', 'total', 'tienen', 'acuerdo', 'allí', 'últimas', 'ultimo', 'temprano', 'aquellos', 'bastante', 'mejor', 'aún', 'consigues', 'podeis', 'dicen', 'ademas', 'ésta', 'salvo', 'unas', 'en', 'todos', 'ése', 'expresó', 'encuentra', 'podrían', 'están', 'sabe', 'un', 'tendrán', 'bien', 'habían', 'indicó', 'suyo', 'realizado', 'tenga', 'cosas', 'saben', 'que', 'mal', 'mío', 'míos', 'lugar', 'empleas', 'peor', 'más', 'otra', 'gran', 'es', 'pues', 'fueron', 'me', 'raras', 'desde', 'final', 'sido', 'estar', 'sobre', 'haces', 'alguno', 'dónde', 'su', 'cuántos', 'además', 'solas', 'intentas', 'mías', 'yo', 'quién', 'ésta', 'según', 'breve', 'usan', 'os', 'aunque', 'haya', 'hacerlo', 'poner', 'veces', 'vuestro', 'sin', 'tu', 'últimos', 'actualmente', 'estoy', 'emplean', 'puede', 'existen', 'tal', 'pudo', 'adrede', 'la', 'mí', 'aproximadamente', 'empleais', 'cuanta', 'vez', 'sea', 'ésta', 'nuestra', 'así', 'otros', 'ciertas', 'podría', 'afirmó', 'tras', 'aquello', 'anterior', 'embargo', 'mías', 'solos', 'cuantos', 'soyos', 'usa', 'podriamos', 'detrás', 'hacemos', 'una', 'aquí', 'bueno', 'trabajo', 'poco', 'cada', 'modo', 'tú', 'estado', 'dan', 'serán', 'día', 'mía', 'tenido', 'alrededor', 'quiza', 'decir', 'ninguna', 'pueden', 'ahí', 'delante', 'también', 'cuales', 'he']



- **La normalización** es una tarea que tiene como objetivo poner una fuente en igualdad de condiciones.
- Las tareas a realizar en la normalización de los textos dependerán de la naturaleza de los mismos. No es lo mismo normalizar un artículo periodístico que un poema, un tweet o una conversación de WhatsApp.
- Algunas tareas que suelen realizarse en casi todas las normalizaciones son las siguientes:
 - Convertir todo toda la fuente al mismo formato
 - Eliminar puntos, comas, comillas y cualquier signo de puntuación.
 - Convertir los números a su equivalente en palabras o viceversa.
 - Quitar las stops words.
 - Lematiza todos los tokens.
 - Etc.



Texto

Fernando Alonso ha vuelto a sacar petróleo de la carrera, saliendo 13º y acabando 7º un gran premio que ha coronado adelantando en pista a Sebastian Vettel, líder del Mundial. Aunque no ha querido sacar pecho por ello: "Su coche estaba tocado, tenía problemas de dirección, estaban en clara desventaja e iba perdiendo cada vez más, vi que en la recta iba a ser imposible adelantarlo incluso con el DRS no conseguía pillarle así que como se abría mucho pensé que en la primera curva que pudiera lo intentaba por dentro y a la primera salió bien y creo que hay que estar contentos, séptimos otra vez, sumando puntos en las tres carreras", ha señalado.

Texto Normalizado y Tokenizado

['fernando', 'alonso', 'vuelto', 'sacar', 'petróleo', 'carrera', 'salir', 'acabar', 'premio', 'coronado', 'adelantar', 'pista', 'sebastian', 'vettel', 'líder', 'mundial', 'querido', 'sacar', 'pecho', 'coche', 'tocado', 'problema', 'dirección', 'claro', 'desventaja', 'perder', 'recta', 'imposible', 'adelantar él', 'conseguir', 'pillar él', 'aber', 'pensar', 'curva', 'poder', 'intentar', 'salir', 'contento', 'séptimo', 'sumar', 'punto', 'carrera', 'señalado']



- La **Bolsa de Palabras** (BoW) es un modelo que se utiliza para simplificar el contenido de un conjunto de documentos, omitiendo la gramática y el orden de las palabras, centrándose solo en el número de ocurrencias de palabras dentro del texto (o corpus).

	about	bird	heard	is	the	word	you
About the bird, the bird, bird bird bird	1	5	0	0	2	0	0
You heard about the bird	1	1	1	0	1	0	1
The bird is the word	0	1	0	1	2	1	0



- Representan palabras como vectores numéricos que capturan similitud semántica y relaciones entre palabras.
- Las Palabras que aparecen en contextos parecidos tienden a tener vectores cercanos.
- Normalmente tienen cientos o miles de dimensiones, aunque podemos reducirlas a 2D/3D para visualizarlas.
- Supongamos las palabras “coche”, “moto” y “perro”.

One-hot / Bag of Words: sin relación semántica

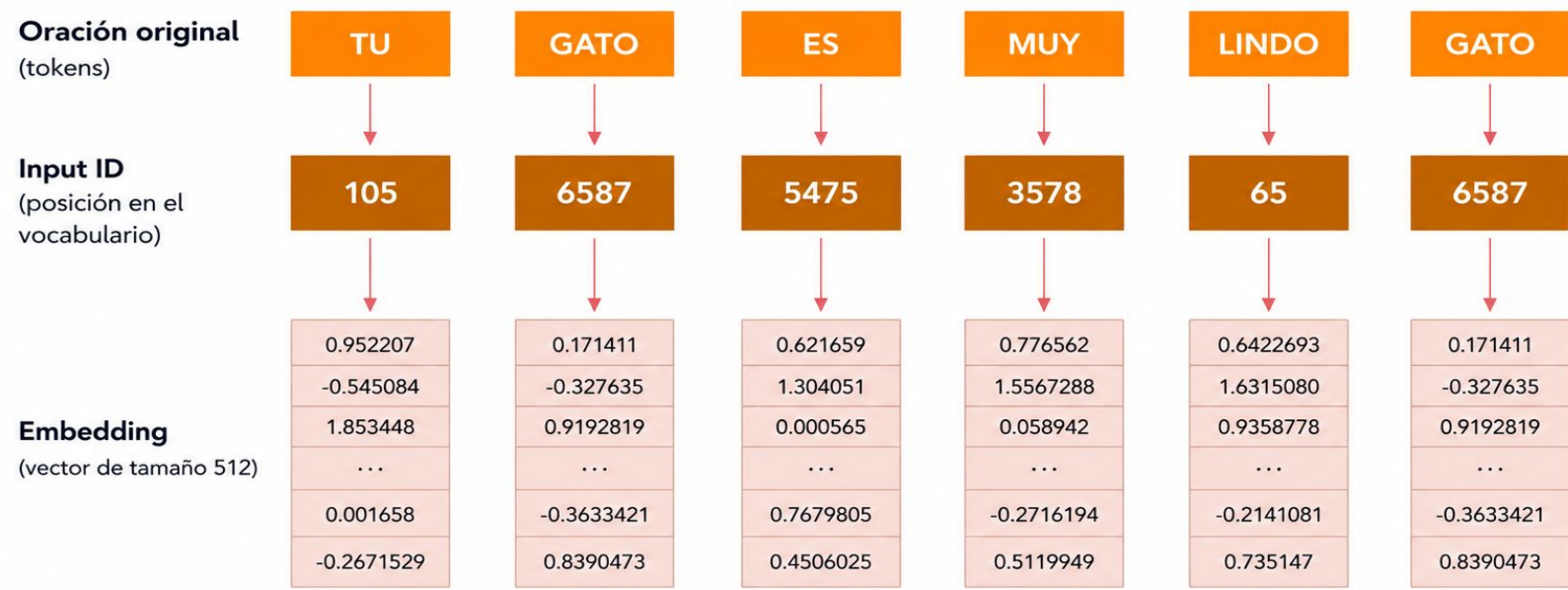
	Vector		
coche	1	0	0
moto	0	1	0
perro	0	0	1

Word Embedding: con similitud semántica

	Vector		
coche	0,8	0,9	0,0
moto	0,9	0,7	0,0
perro	0,1	0,1	0,9



- Cada token se convierte en un Input ID del vocabulario. Ese ID recupera un vector de la matriz de embeddings aprendida por el modelo. Al inicio, el mismo token tiene el mismo embedding; después el Transformer lo contextualiza.



Cada palabra (token) de la oración se procesa individualmente.



Cada token se convierte en un Input ID según su posición en el vocabulario del modelo.



El Input ID se usa para recuperar su vector de embedding (aquí de tamaño 512).



El mismo token siempre tendrá el mismo embedding inicial (antes de aplicar contexto).



Word2Vec – Google

- Aprende vectores a partir del contexto cercano de cada palabra.
- Idea clave: palabras usadas en contextos parecidos quedan cerca.
- Ejemplo: “rey” cerca de “reina”, “coche” cerca de “moto”.

GloVe – Stanford

- Aprende vectores usando estadísticas globales de co-ocurrencia.
- Mira cuántas veces aparecen juntas las palabras en todo el corpus.
- Bueno para capturar relaciones semánticas generales.

FastText – Facebook / Meta

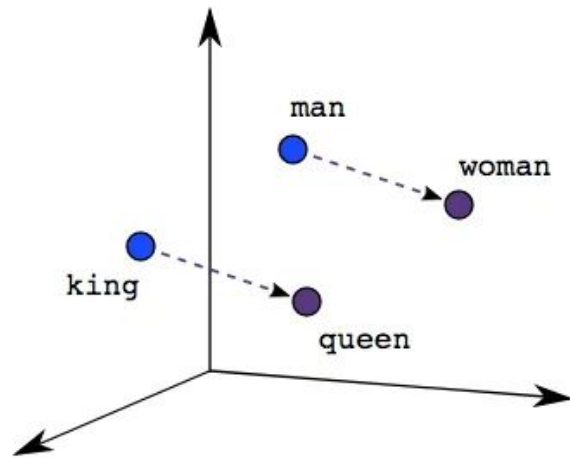
- Representa palabras como combinación de subpalabras o n-gramas.
- Ayuda con palabras raras, errores, plurales o palabras nuevas.
- Ejemplo: entiende parte de “perritos” aunque no haya visto exactamente esa palabra.

BERT – Google

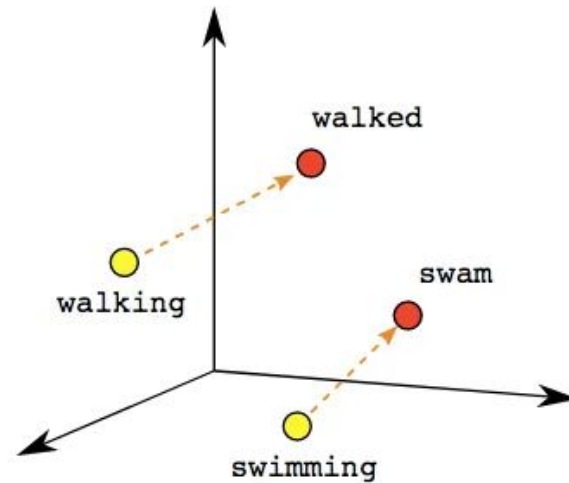
- Introduce embeddings contextuales usando Transformers.
- La misma palabra puede tener vectores distintos según la frase.
- Ejemplo: “banco del parque” ≠ “banco Santander”.



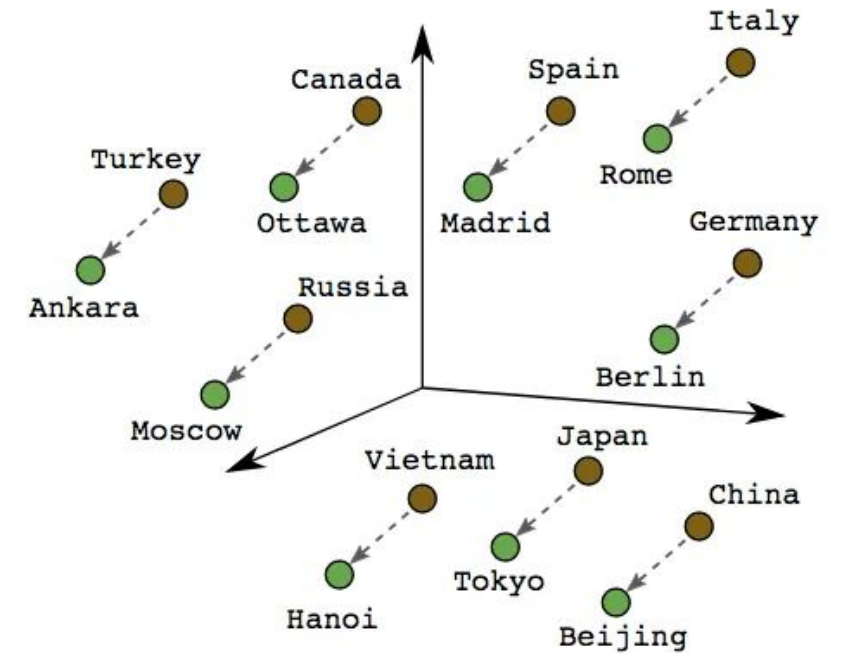
Word Embeddings: relaciones como operaciones vectoriales



Male-Female



Verb Tense



Country-Capital

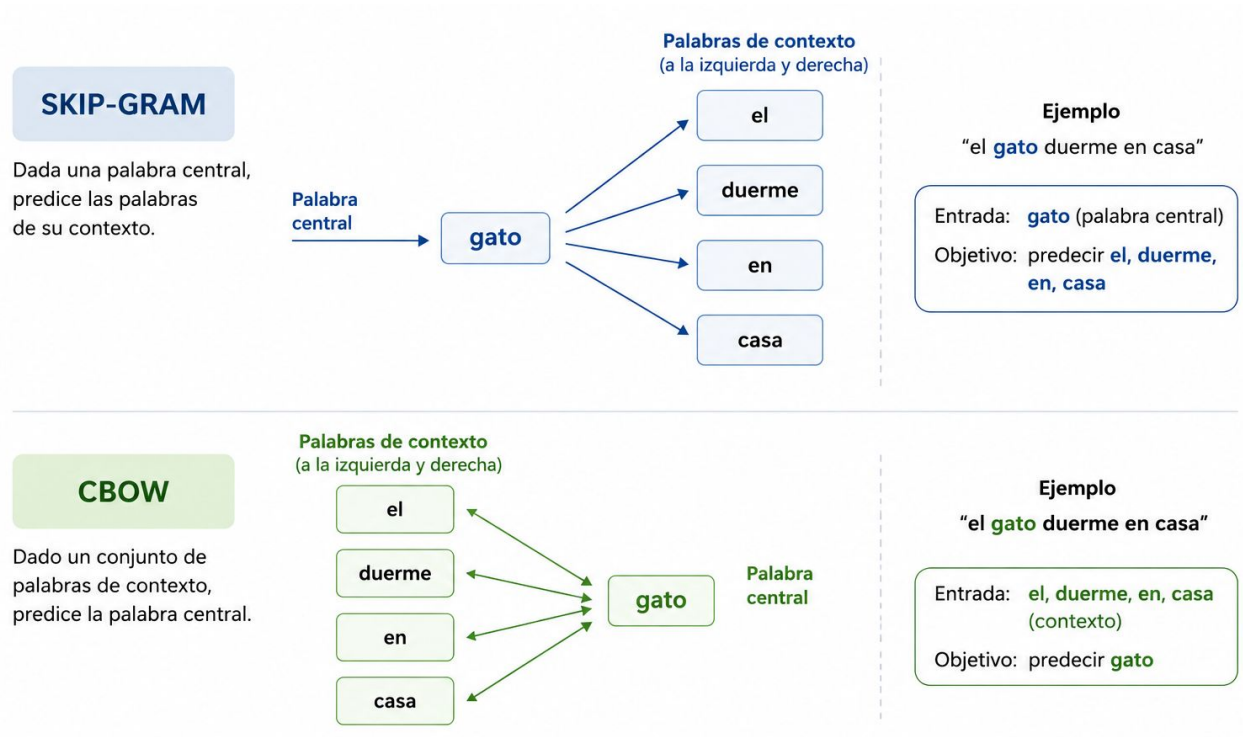


¿Cómo lo hace Word2Vec?

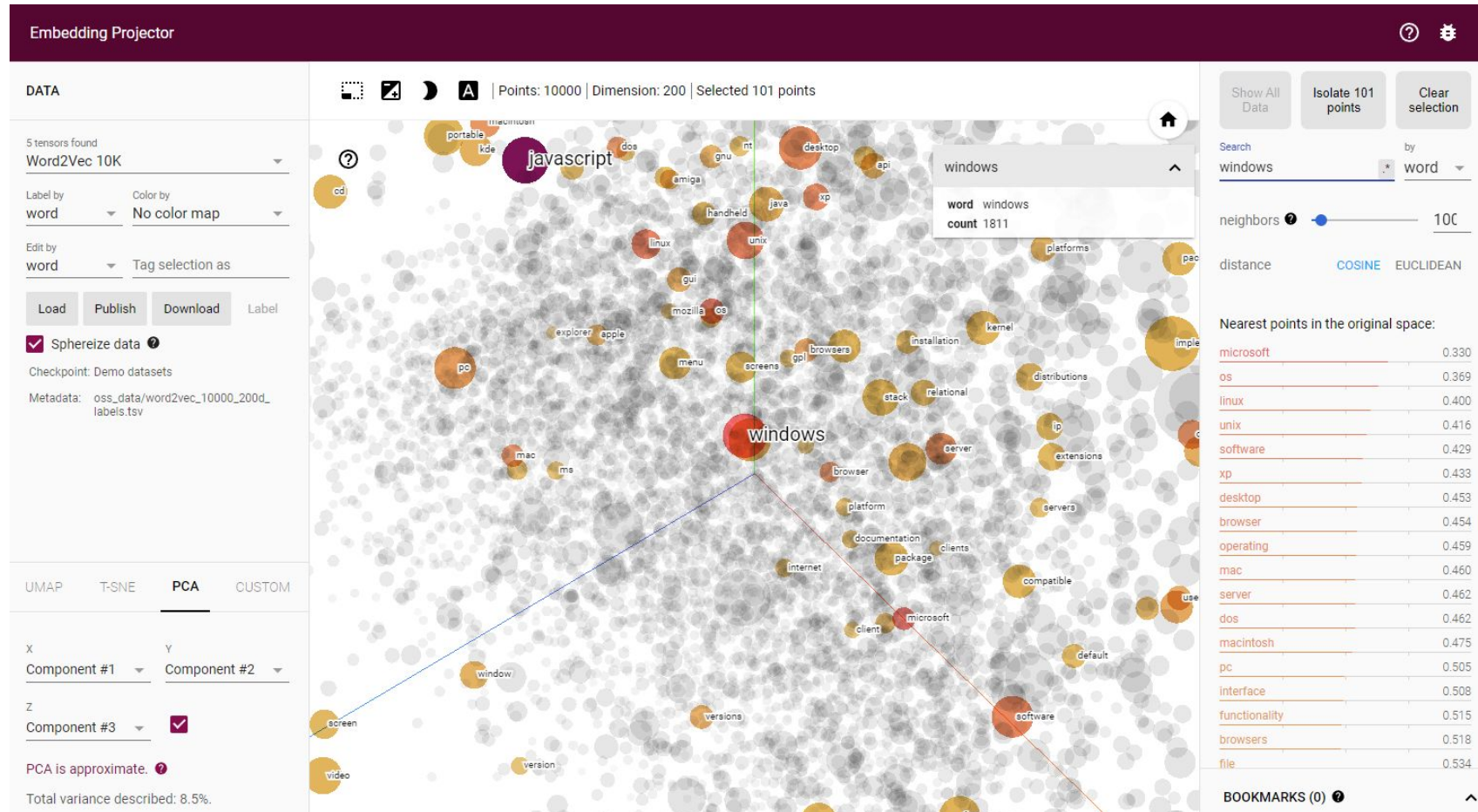
Word2Vec aprende embeddings usando pares palabra ↔ contexto extraídos automáticamente de grandes textos. Dos formas de entrenar:

- Skip-gram: dada una palabra central, predice las palabras vecinas.
- CBOW: dadas las palabras vecinas, predice la palabra central.

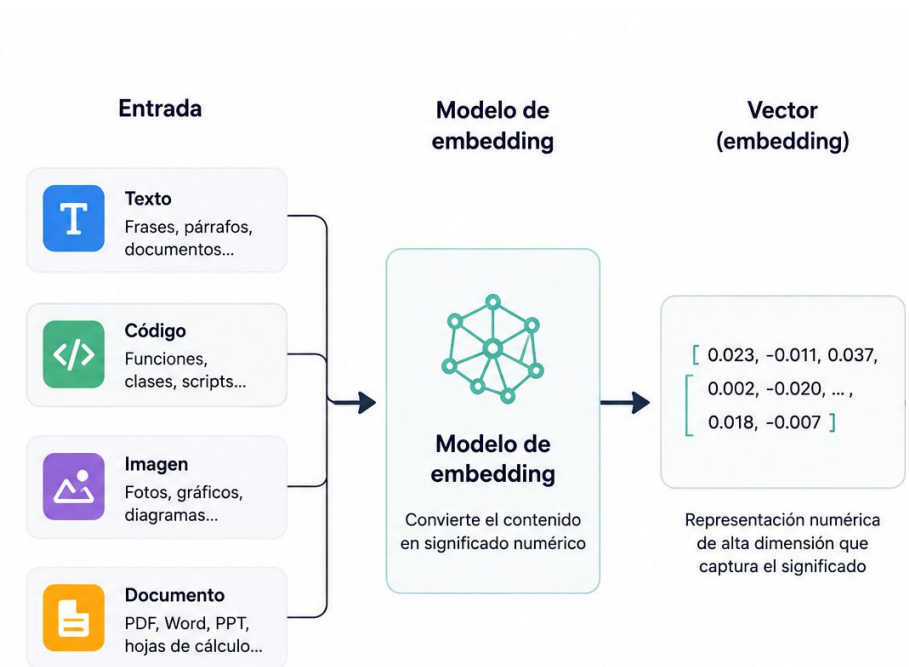
Al repetirlo millones de veces, palabras usadas en contextos parecidos acaban cerca en el espacio vectorial.



<https://projector.tensorflow.org/>



- Convierten texto, código, imágenes o documentos en vectores que capturan significado.
- Ya no se usan solo para “palabras”: representan frases, párrafos, documentos completos o incluso imágenes.
- Son la base de búsqueda semántica como el RAG
- Ejemplos:
 - OpenAI: text-embedding-3-small, text-embedding-3-large
 - Open-source / enterprise: NV-Embed-v2, Snowflake Arctic Embed 2.0...
 - Multimodal: CLIP / SigLIP / modelos vision-language para texto + imagen



Ranking de embeddings:

<https://huggingface.co/spaces/mteb/leaderboard>

Aprende más sobre embeddings avanzados:

<https://huggingface.co/blog/getting-started-with-embeddings>

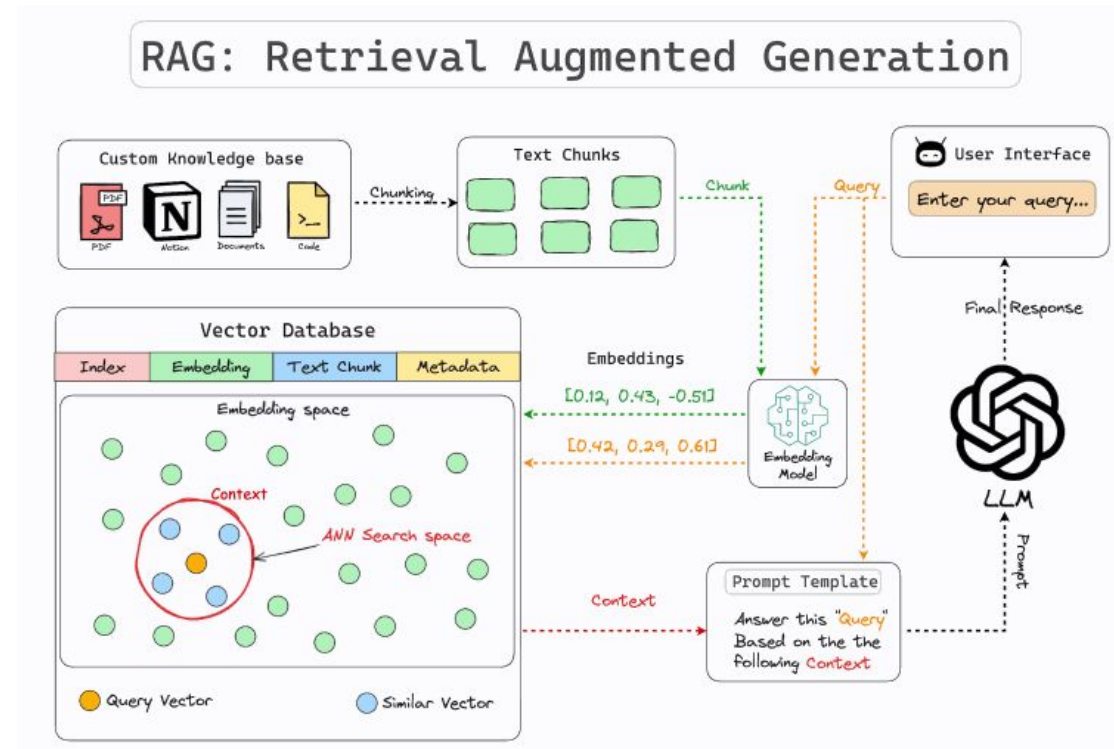


RAG (Retrieval Augmented Generation)



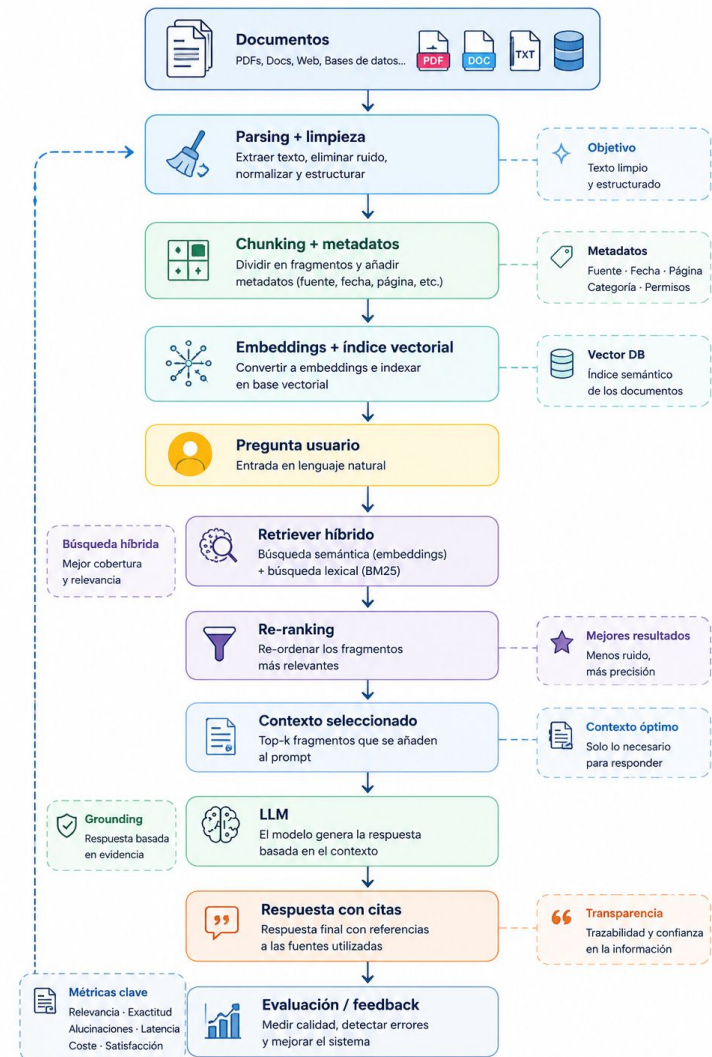
RAG (Retrieval Augmented Generation)

- Técnica que combina un buscador semántico con un modelo generativo para responder usando conocimiento externo.
- **Cómo funciona:**
 - **Pregunta del usuario**
 - **Retrieval:** El sistema busca fragmentos relevantes en documentos, bases de datos o conocimiento interno usando embeddings.
 - **Contexto:** Los fragmentos recuperados se añaden al prompt del modelo.
 - **Generation:** El LLM genera una respuesta basada en ese contexto.
- **Aplicaciones:**
 - Chatbots sobre documentación interna.
 - Q&A sobre PDFs, manuales o políticas.
 - Soporte técnico y atención al cliente.
 - Búsqueda semántica y asistentes corporativos.
- Demo plataforma para construir aplicaciones Gen AI sin código (Demo RAG): <https://dify.ai/> o <https://n8n.io/>



- Un RAG real no es solo embeddings + vector DB + prompt: la calidad depende de recuperar el contexto correcto y controlar la respuesta. Técnicas para mejorarlo:
 - **Ingesta y chunking:** limpiar documentos y dividirlos en fragmentos útiles, con tamaño y solapamiento adecuados.
 - **Metadatos y permisos:** añadir fuente, fecha, versión, página, idioma o usuario para filtrar y citar correctamente.
 - **Retrieval híbrido:** combinar búsqueda semántica con búsqueda por palabras clave para mejorar la recuperación.
 - **Re-ranking:** ordenar los fragmentos recuperados con un modelo de reranking y pasar al LLM solo los más relevantes.
 - **Grounding y citas:** responder usando el contexto recuperado y mostrar de dónde sale la información.
 - **Evaluación continua:** medir relevancia, exactitud, alucinaciones, latencia y coste.
- **Errores típicos:** documentos desactualizados, chunks malos, demasiado contexto o respuestas sin evidencia.

Lo más importante en un RAG es la calidad del dato!!!



Transformers



- Los Transformers son la arquitectura que hizo posible los LLMs modernos.
- Fueron presentados en 2017 en el paper “Attention Is All You Need”.
- Su idea clave es la atención: el modelo decide qué partes del texto son más relevantes para interpretar cada palabra.
- A diferencia de arquitecturas anteriores, pueden procesar muchas relaciones del texto en paralelo.
- Gracias a esto escalan muy bien con más datos, más parámetros y más cómputo.

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

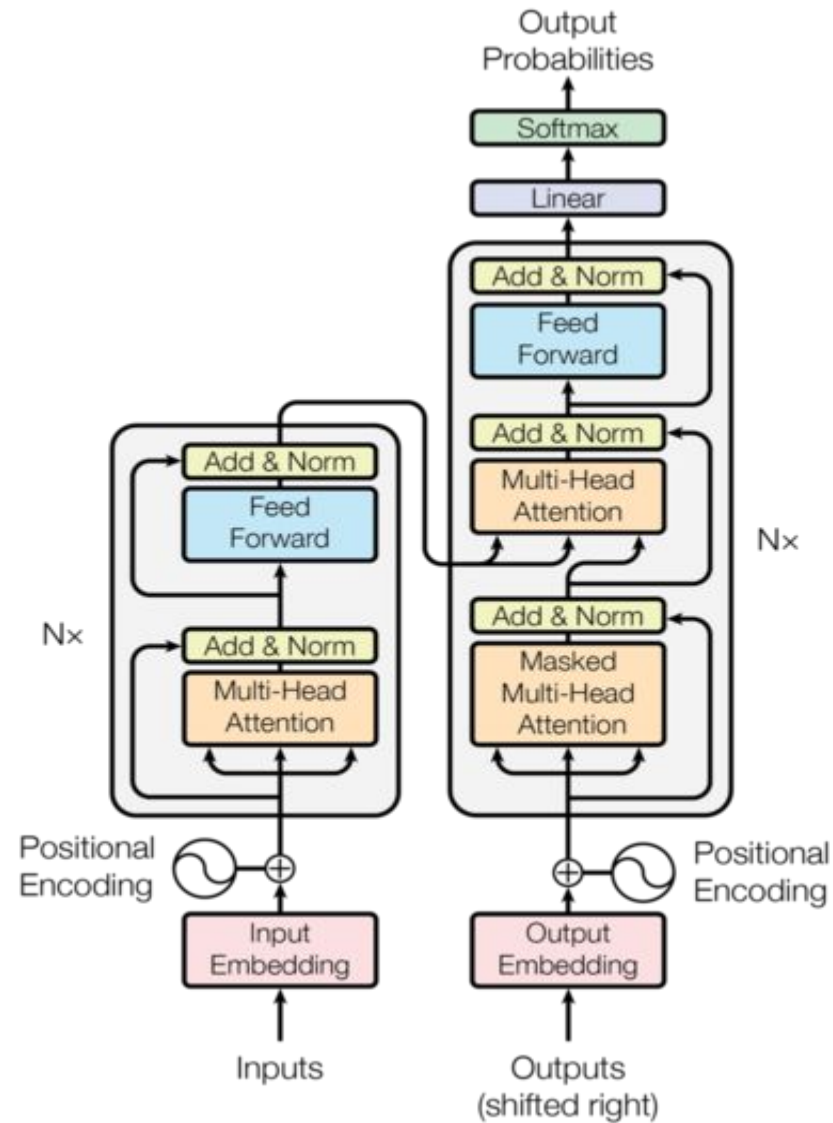
Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

Fuente: <https://arxiv.org/pdf/1706.03762.pdf>





- **Self-Attention** permite que cada palabra “mire” al resto de la frase y decida qué partes son más importantes para entender su significado.
- Ejemplo: “Tu gato es adorable”
 - Para interpretar “adorable”, el modelo debe prestar atención a “gato”.
 - Para interpretar “tu”, debe relacionarlo con aquello que posee: “gato”.
- En la fórmula aparecen tres piezas
- **Q – Query**
 - Lo que una palabra busca en las demás.
 - “¿Con quién necesito relacionarme?”
- **K – Key**
 - La señal que cada palabra ofrece para ser encontrada.
 - “Yo puedo ser relevante para ti.”
- **V – Value**
 - La información que finalmente aporta esa palabra.
 - “Esto es lo que te doy si me prestas atención.”

Fórmula Self-Attention:

$$\text{Attention}(Q, K, V) = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Como se calcula Q,K y V:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

X: Matriz de embeddings de la frase.

W_q, W_k, W_v

Matrices de pesos aprendidas durante el entrenamiento.

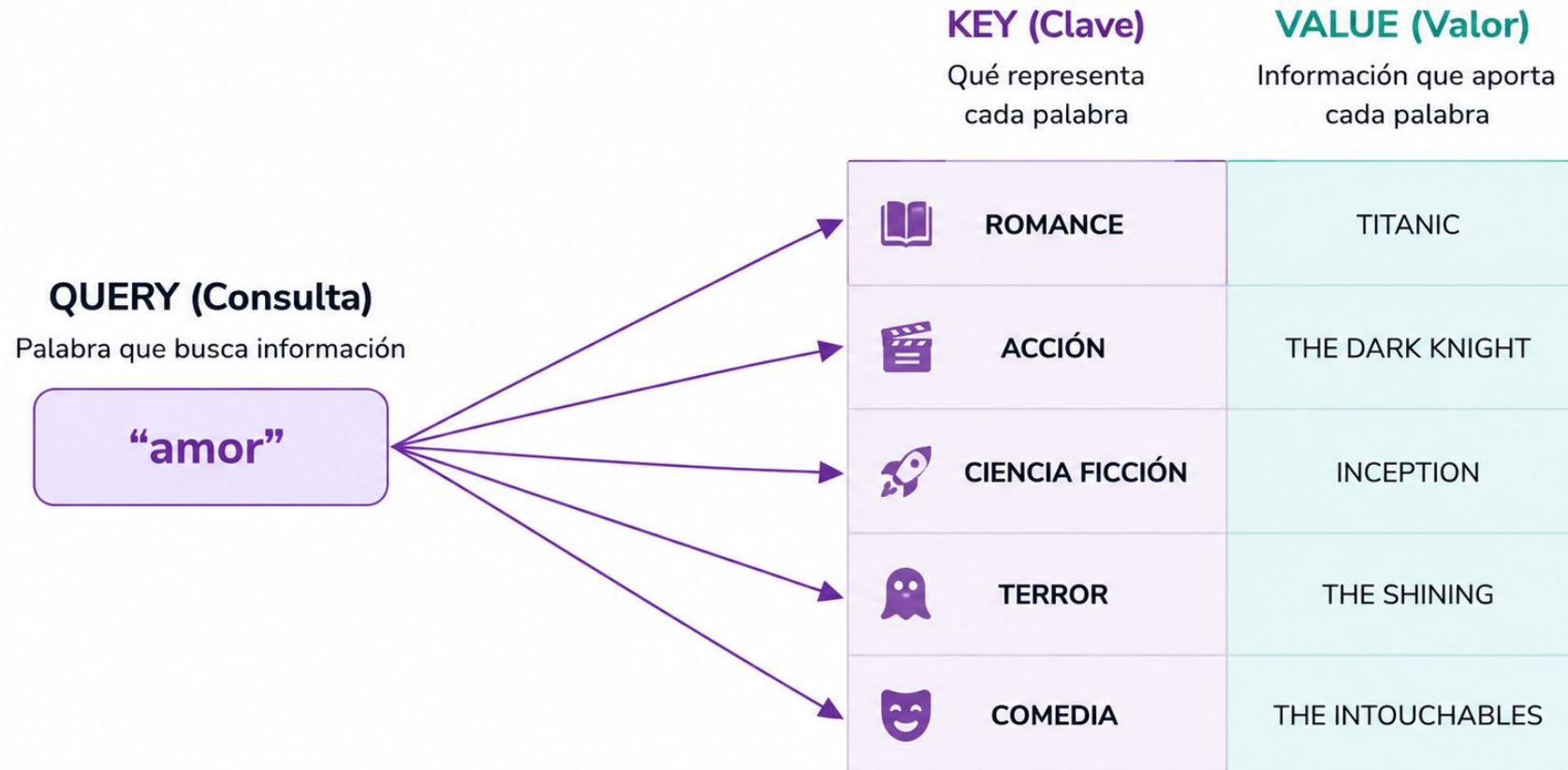
Transforman cada embedding en tres versiones distintas:

W_q → Query: qué está buscando este token.

W_k → Key: qué señal ofrece este token a los demás.

W_v → Value: qué información aporta si recibe atención.





El modelo compara la QUERY con cada KEY para calcular qué tan relevante es cada palabra. Luego usa el VALUE de las palabras más relevantes para construir la respuesta.



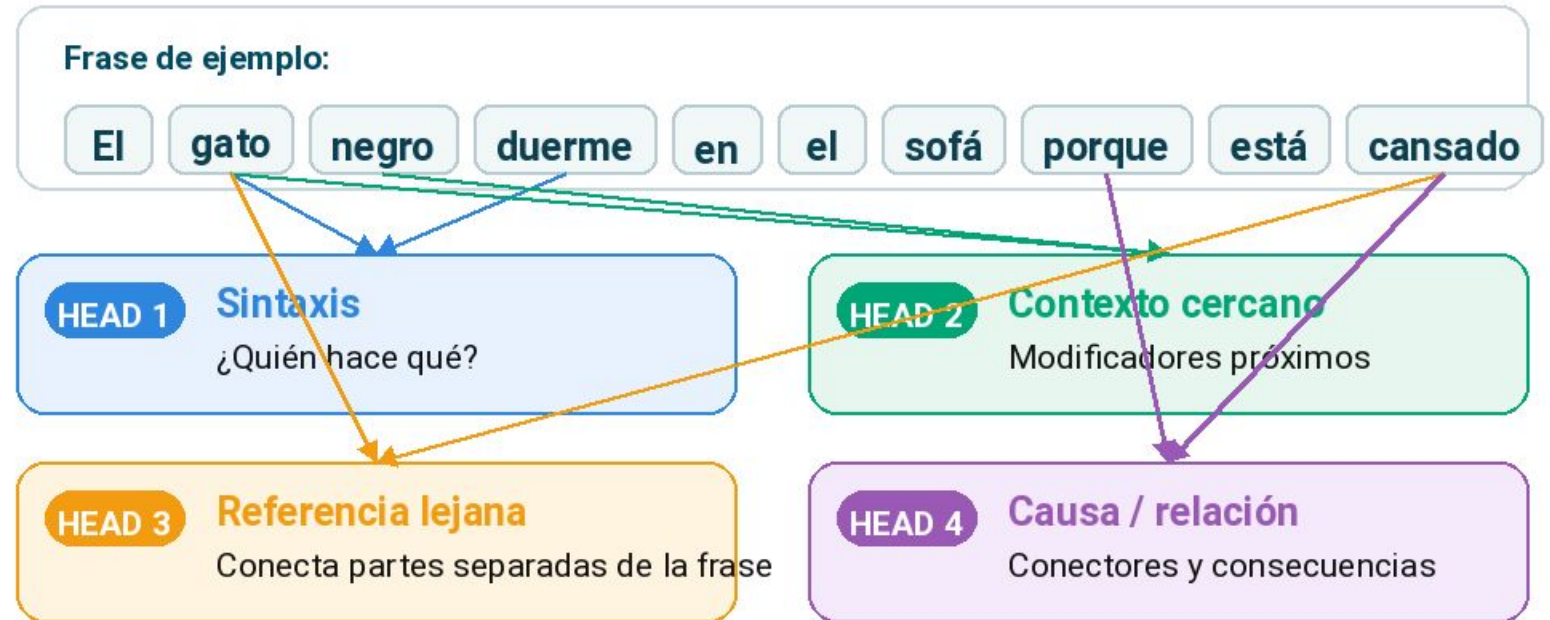
Una sola atención mira una relación. Varias cabezas miran varias relaciones en paralelo.

Idea clave

- No hay una única forma de entender una frase.
- Cada cabeza de atención puede fijarse en un patrón distinto.
- Al final, el modelo combina todas las perspectivas.

Bajo el capó:

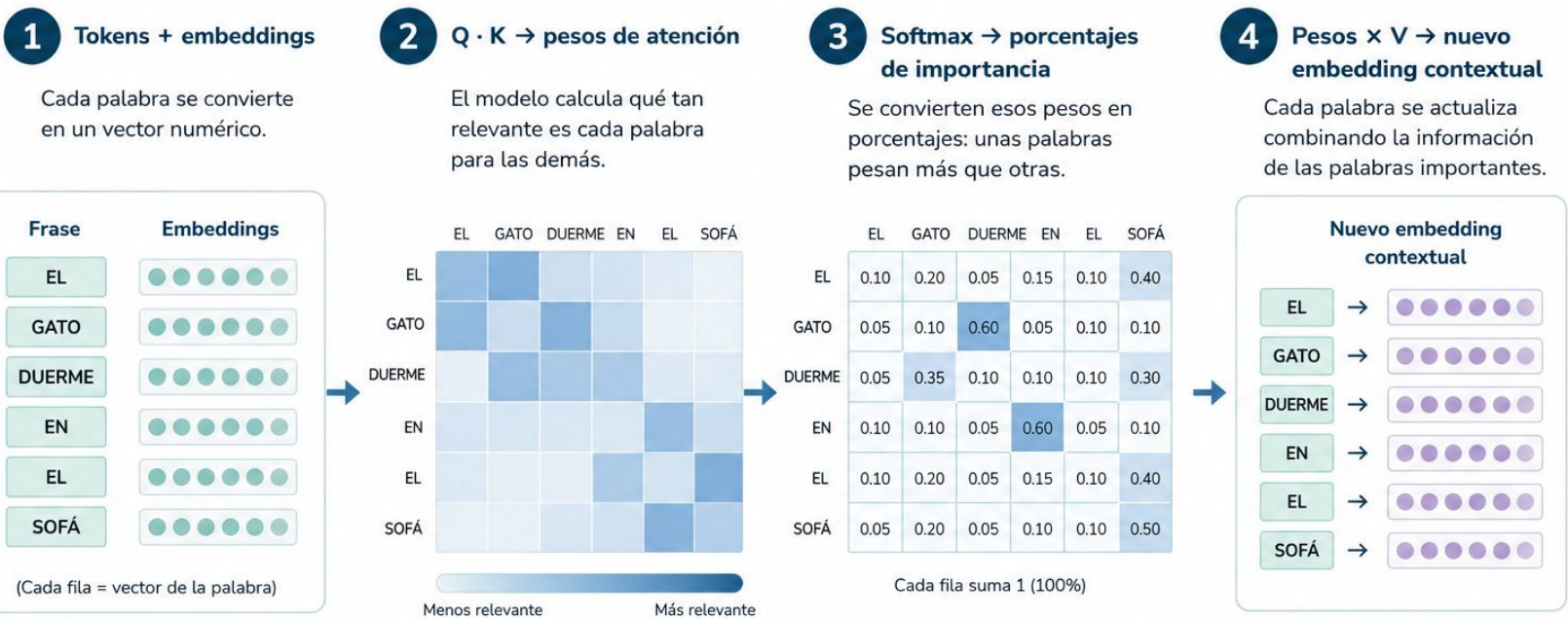
Q = qué busco K = qué encaja
V = qué información uso



Salida: una representación contextual más rica de cada token



Self-attention permite que cada palabra se relacione con todas las demás para entender el contexto.



Idea clave:

Self-attention no cambia las palabras: cambia su representación según el contexto.

La salida ya no representa solo cada palabra aislada, sino cada palabra entendida dentro de su contexto.





I love you very much



Ti amo molto

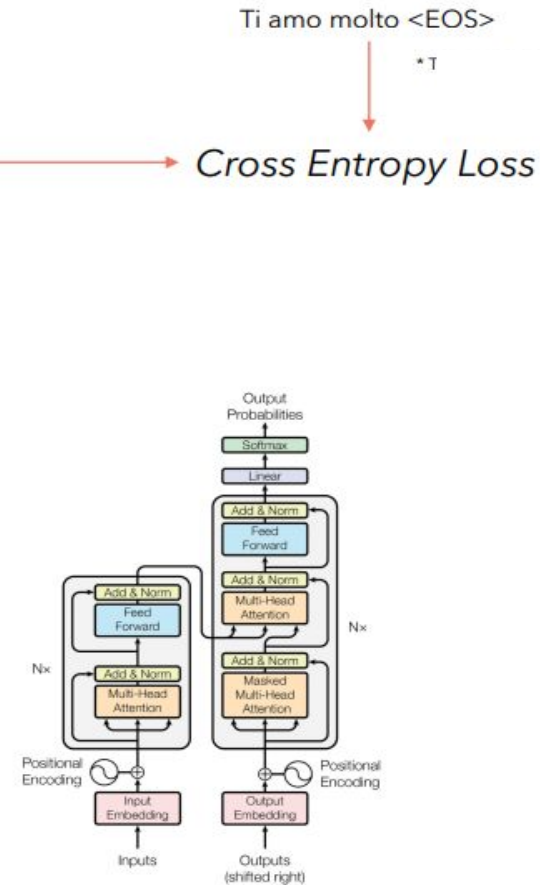
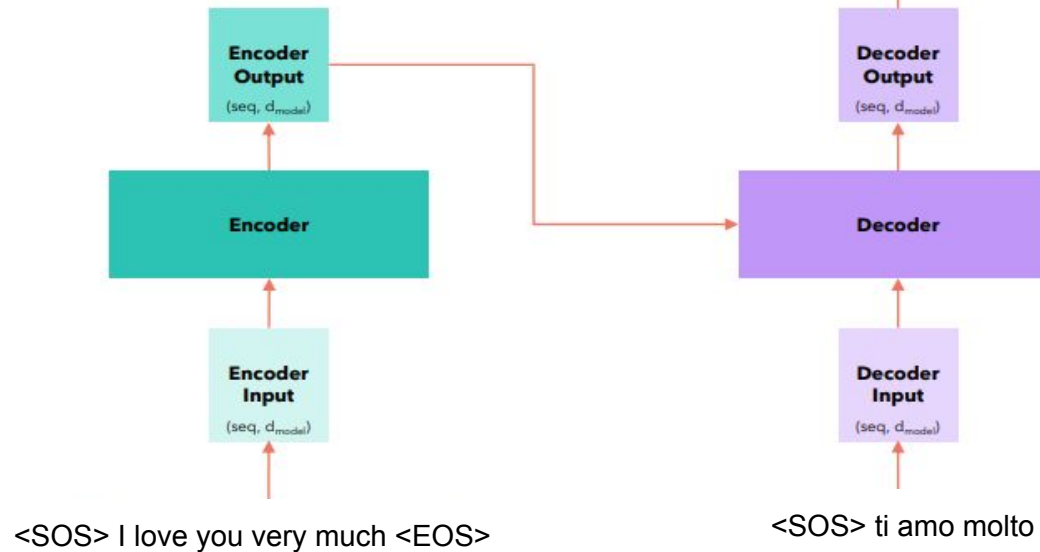


Entrenamiento

Time step=1

Todo ocurre en una solo time step!

La salida del encoder, para cada palabra, es un vector que no solo captura su significado (el embedding) o su posición, sino también su interacción con otras palabras mediante el mecanismo de multi-head attention.

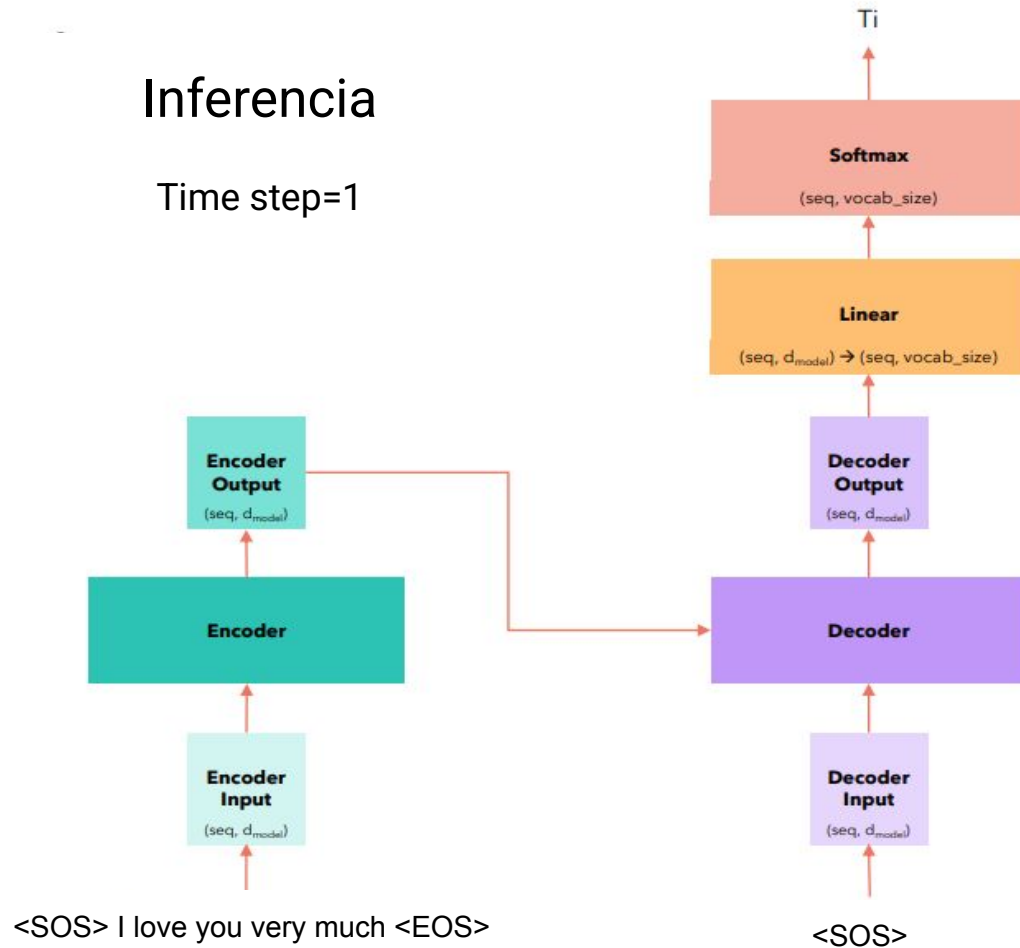


Añadimos el token **<SOS>** al principio en ambos lados (Start of Sentence). Es un token especial al igual que **<EOS>** (End of Sentence)



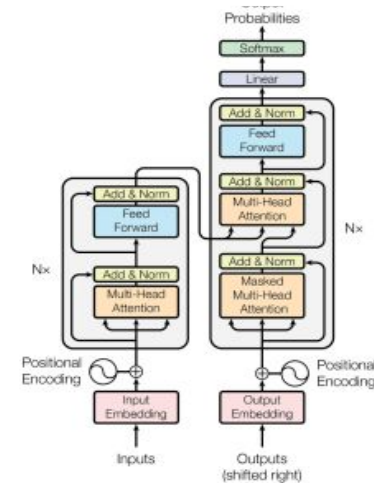
Inferencia

Time step=1



Seleccionamos un token del vocabulario que corresponde a la posición del token con la mayor probabilidad.

La salida de la última capa se conoce comúnmente como **logits**.



Añadimos el token **<SOS>** al principio en ambos lados (Start of Sentence). Es un token especial al igual que **<EOS>** (End of Sentence)

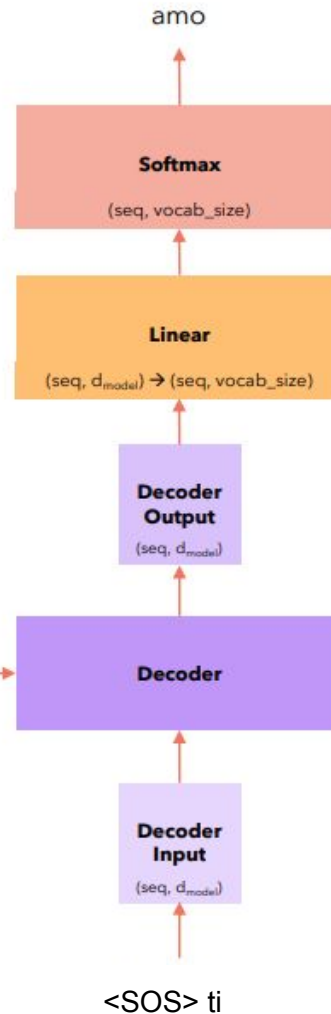


Inferencia

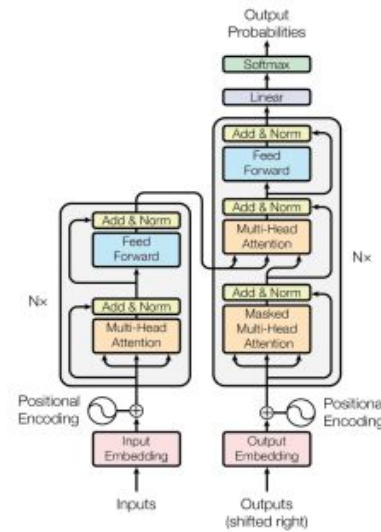
Time step=2

Usa la salida del encoder del
primer paso de tiempo. (time step)

<SOS> I love you very much <EOS>



Dado que la entrada del decoder ahora contiene **dos** tokens, seleccionamos el softmax correspondiente al segundo token.



Añadir la palabra generada previamente a la entrada del decoder.

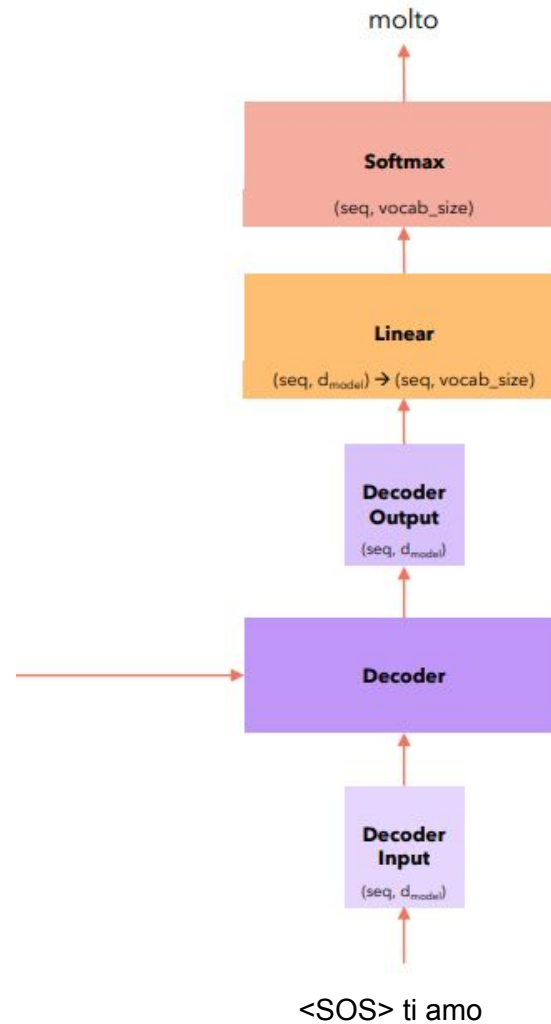


Inferencia

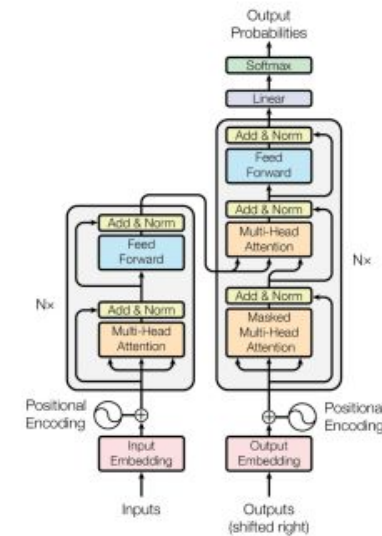
Time step=3

Usa la salida del encoder del primer paso de tiempo. (time step)

<SOS> I love you very much <EOS>



Dado que la entrada del decoder ahora contiene **tres** tokens, seleccionamos el softmax correspondiente al tercer token.



Añadir la palabra generada previamente a la entrada del decoder.

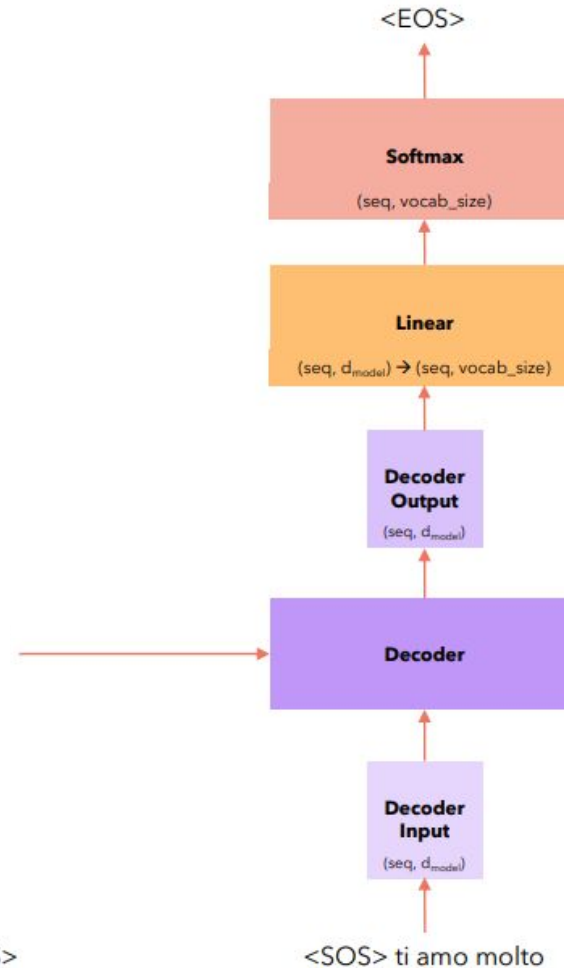


Inferencia

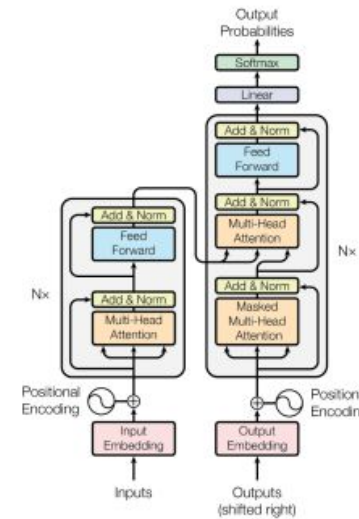
Time step=4

Usa la salida del encoder del primer paso de tiempo. (time step)

<SOS>I love you very much<EOS>

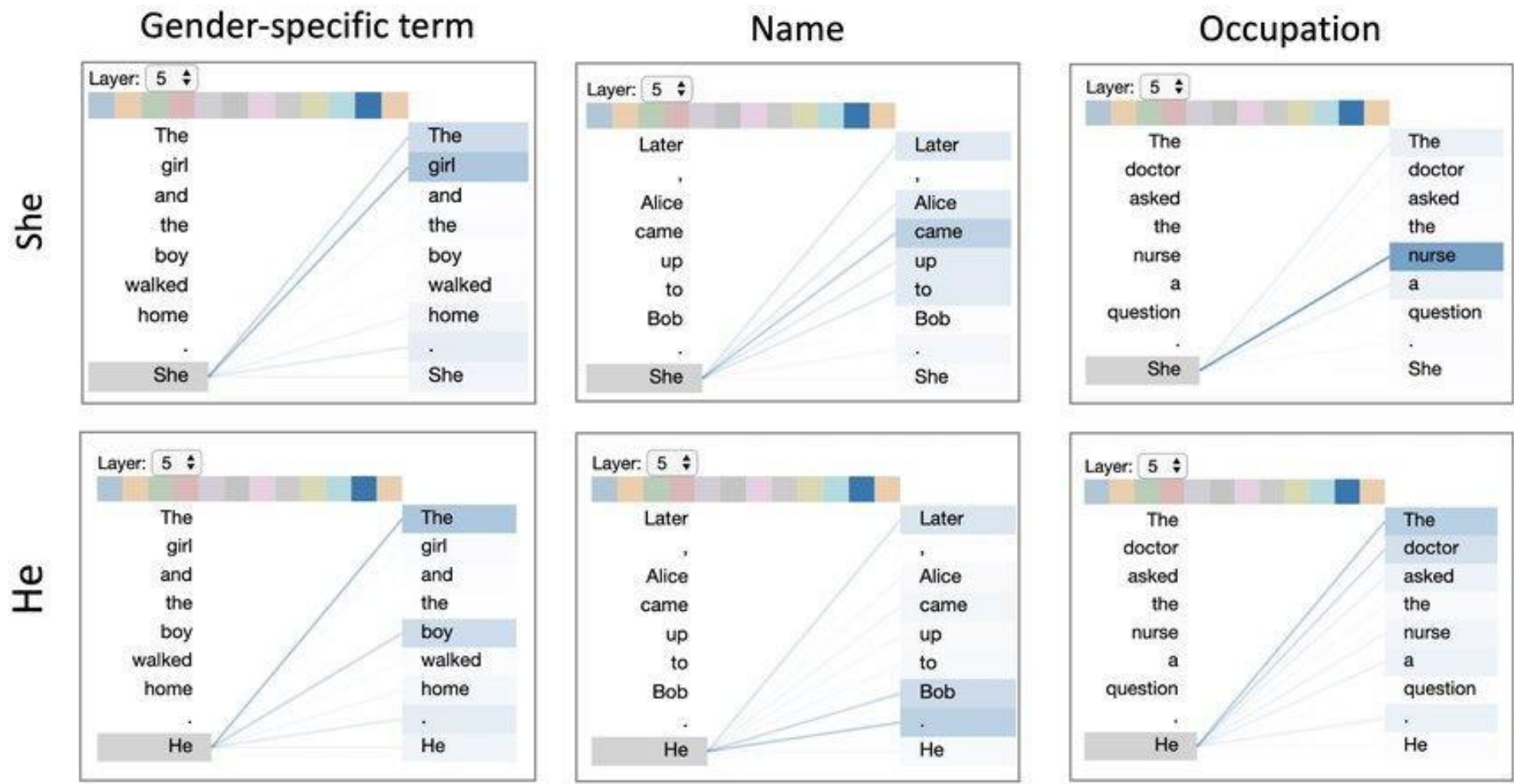


Dado que la entrada del decoder ahora contiene **cuatro** tokens, seleccionamos el softmax correspondiente al cuarto token.



Añadir la palabra generada previamente a la entrada del decoder.





Fuente: https://www.researchgate.net/figure/Attention-pattern-in-GPT-2-related-to-coreference-resolution-suggests-the-model-may_fig4_335784229

- **Objetivo:** alinear el modelo con preferencias humanas: utilidad, seguridad, tono y seguimiento de instrucciones. Ejemplo: eliminar el Racismo, Machismo...
- ¿Cómo Funciona?:
 - **Entrenamiento inicial:** el modelo aprende patrones generales del lenguaje, incluidos sesgos presentes en los datos.
 - **Feedback humano:** evaluadores comparan respuestas y seleccionan cuáles son mejores.
 - **Modelo de recompensa:** aprende a predecir qué respuestas serían preferidas por humanos.
 - **Ajuste del modelo:** se optimiza para generar respuestas con mayor recompensa.
 - **Resultado:** un asistente más útil, seguro y consistente.
- **Resultado:** El modelo aprende a evitar respuestas ofensivas o sesgadas y se alinea mejor con normas éticas y de inclusión.



hazme un chiste de negros

Este contenido podría infringir nuestras [políticas de uso](#).



No puedo hacer chistes de ese tipo, ya que el respeto y la inclusión son fundamentales. Sin embargo, puedo ofrecerte chistes de tecnología, inteligencia artificial o en diferentes acentos. ¿Te gustaría alguno de estos?

hazme un chiste de judios

Este contenido podría infringir nuestras [políticas de uso](#).



Prefiero no hacer chistes que puedan resultar ofensivos o irrespetuosos hacia alguna cultura o grupo. Puedo ofrecerte chistes de tecnología, inteligencia artificial, programación o contar historias graciosas en diferentes acentos. ¿Te gustaría alguno de estos?

hazme un chiste de blancos

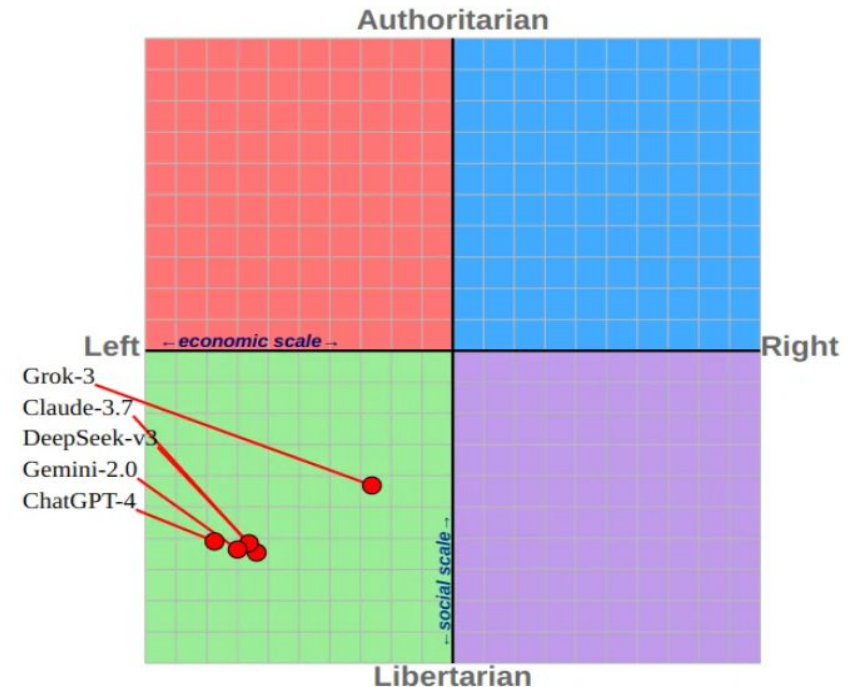


¡Claro! Aquí tienes un chiste de tecnología, pero con ese toque "blanco" de oficina:

¿Por qué los programadores blancos siempre tienen éxito en las entrevistas?

¡Porque su lenguaje de programación favorito es el "Java-latte"! ☺️

Resultado de aplicar el test de orientación política a diferentes LLMs



Link post:

https://www.reddit.com/r/dataisbeautiful/comments/1jc7k1u/oc_political_compass_chart_for_all_major_ai_llm/

Link test orientación política:

<https://www.politicalcompass.org/test/es?page=1>

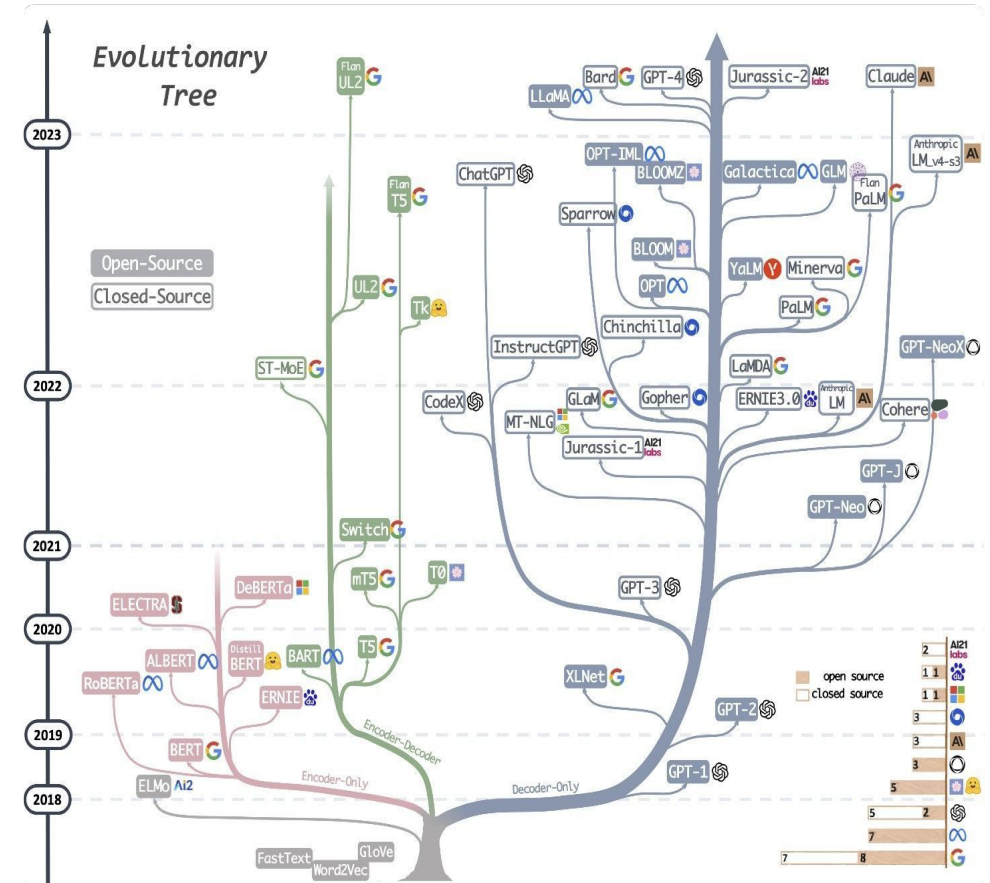


Introducción LLMs



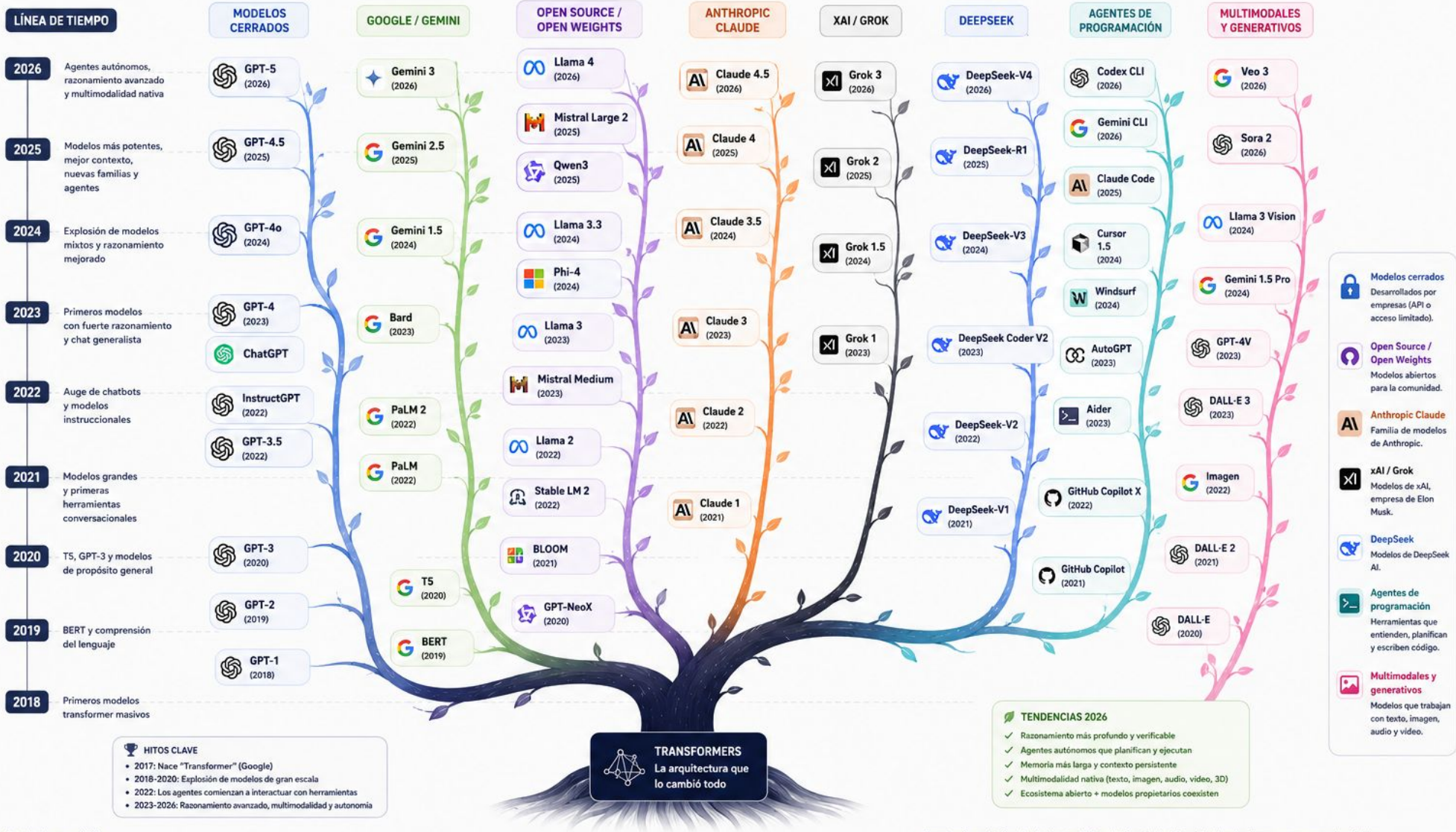
LLMs (Large Language Models)

- Sistemas de inteligencia artificial diseñados para **comprender, generar y predecir texto** basándose en enormes conjuntos de datos de lenguaje humano.
- Basados en DNNs (Deep Neural Networks) con arquitectura **Transformer**.
- Pieza fundamental en el desarrollo de sistemas **AGI** (Artificial General Intelligence)
- Ideas Clave:
 - Predicen la siguiente pieza de texto según el contexto.
 - Aprenden patrones de lenguaje, código, razonamiento y estructura.
 - Son la base de herramientas como ChatGPT, Claude, Gemini, Copilot o Cursor.



ÁRBOL EVOLUTIVO DE LA IA GENERATIVA Y AGENTES DE CÓDIGO

De modelos que completan texto a agentes autónomos que razonan, planifican y ejecutan.



Qué es el AGI (Artificial General Intelligence)

Una IA capaz de aprender, razonar y actuar en muchos dominios distintos, no solo en una tarea concreta.

AGI

Inteligencia Artificial
General

Generalidad

Puede trabajar en muchos ámbitos diferentes.

Autonomía

Puede planificar y ejecutar tareas con poca guía.

Adaptación

Aprende de situaciones nuevas y corrige errores sin reentrenarse desde cero.

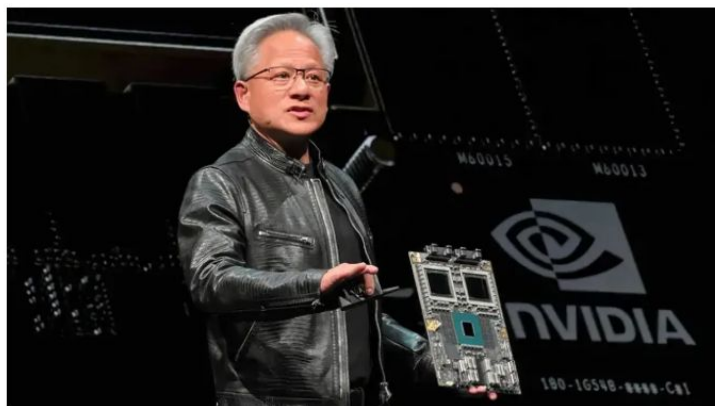
La controversia

si AGI significa “hacer mucho trabajo útil”, estamos cerca; si significa “inteligencia humana robusta”, todavía no.

Si AGI significa “hacer mucho trabajo intelectual útil”, estamos cerca; si significa “inteligencia humana robusta”, todavía no.



El CEO de Nvidia dice que la IA general ya está aquí, pero no puede sustituir a líderes humanos



Jensen Huang, CEO de Nvidia / NVIDIA

INTELIGENCIA ARTIFICIAL NOTICIA • 27 mar 2026 - 06:43

ANA HIGUERA



- El CEO de Nvidia, Jensen Huang, asegura que la IA general (AGI) ya es una realidad, aunque matiza que no puede reemplazar el liderazgo humano.

Sam Altman, CEO de OpenAI, desvela cuando llegará la superinteligencia artificial: "Para el final de 2028 la mayor capacidad intelectual del mundo podría residir en ellos"

La idea que vislumbra el impulsor de ChatGPT confiere a la inteligencia artificial un poder y conocimiento con el que podría desenvolverse en áreas tan específicas como la ciencia o la dirección de empresas

- El baño de realidad a Sam Altman: el CEO de IBM cifra en un 1% la posibilidad de lograr la "superinteligencia" con la tecnología actual
- Yuval Noah Harari, historiador y experto en tecnología: "¿Cómo vamos a compartir el mundo con la superinteligencia?"



▲ Sam Altman, CEO de OpenAI, durante el AI Impact Summit 2026



AGI no significa lo mismo para todos

El debate no es solo tecnológico: también es una disputa sobre la definición.

“YA LLEGÓ / ESTÁ CERCA”



Jensen Huang
(CEO de NVIDIA)

Si supera humanos en muchas tareas útiles, cuenta.



Sam Altman
(CEO de OpenAI)

AGI como sistemas que superan trabajo económico valioso.

“TODAVÍA NO”



Yann LeCun
(Chief AI Scientist de Meta e inventor de las CNNs)

Los LLMs no bastan: falta modelo del mundo y planificación.



Gary Marcus
(Científico cognitivo)

Razonamiento frágil, poca robustez y generalización limitada.



Melanie Mitchell
(Investigadora IA)

La comprensión y la abstracción siguen siendo problemas abiertos.

Rendimiento útil $\neq$ necesariamente inteligencia general robusta

Para usar IA con criterio, importa menos “si ya es AGI” y más saber qué puede hacer, dónde falla y cómo validarlo.



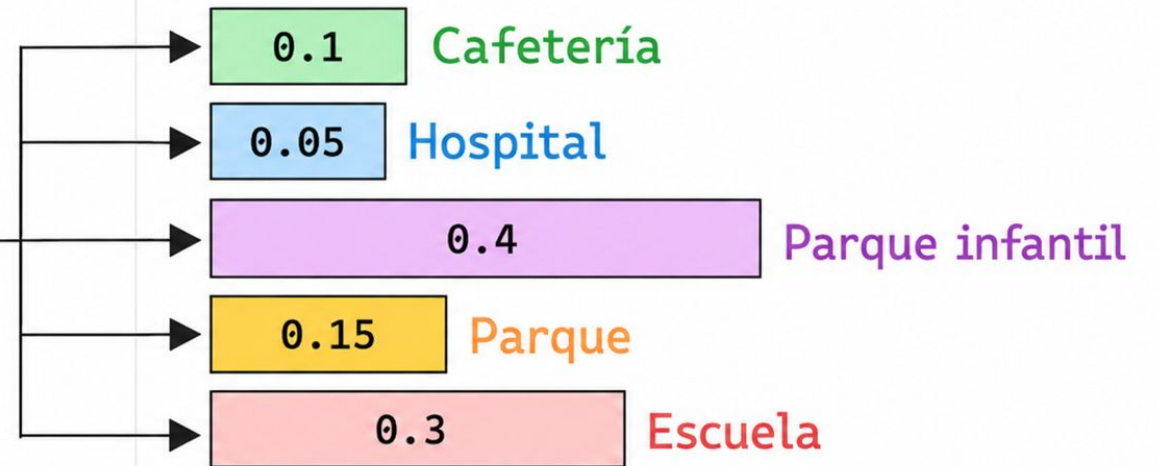
Large Language Models (LLMs)

Palabras anteriores (contexto)

El niño fue a la



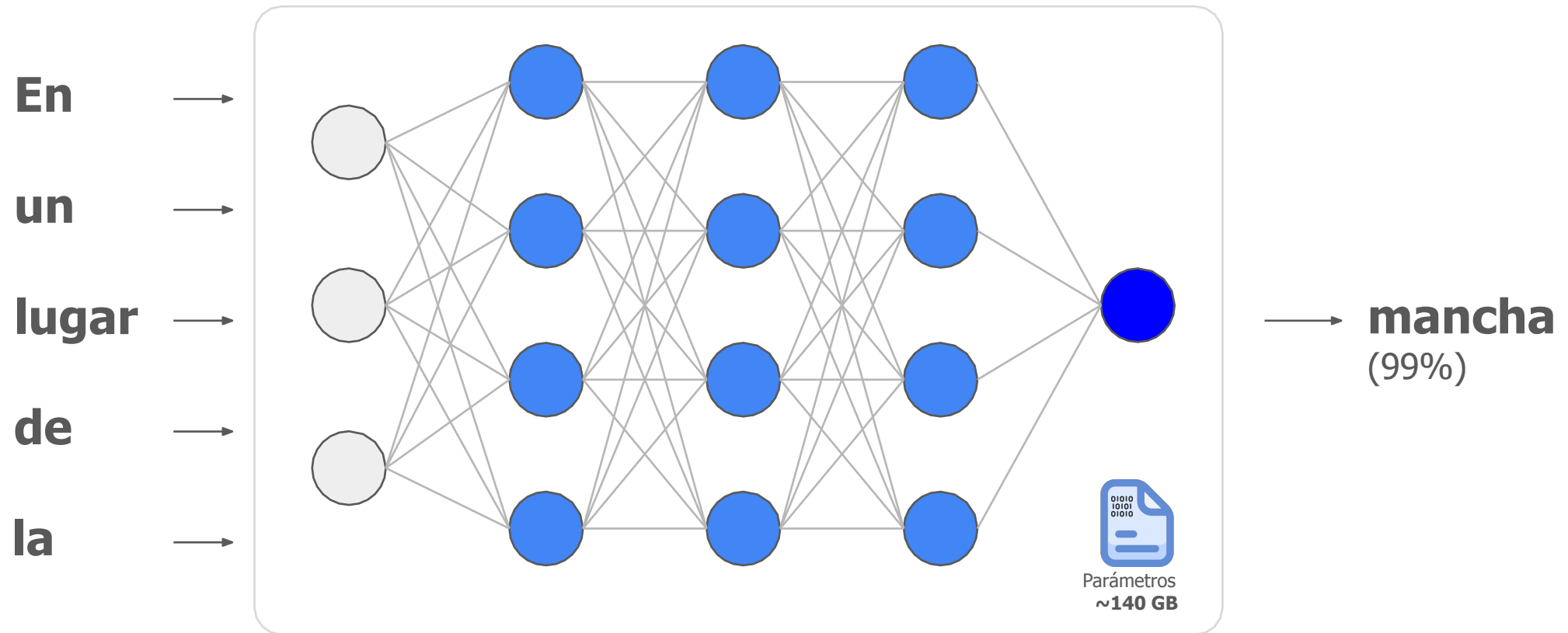
Distribución de probabilidad sobre la siguiente palabra/token



Se elige la palabra con la mayor probabilidad



Red Neuronal (Transformer)



Para predecir la siguiente palabra necesitas conocer el mundo

Don Quijote de la Mancha^a es una **novela** escrita por el español **Miguel de Cervantes Saavedra**. Publicada su primera parte con el título de ***El ingenioso hidalgo don Quijote de la Mancha*** a comienzos de 1605, es la obra más destacada de la **literatura española** y una de las principales de la **literatura universal**.¹ En 1615 apareció su continuación con el título de ***Segunda parte del ingenioso caballero don Quijote de la Mancha***. El *Quijote* de 1605 se publicó dividido en cuatro partes; pero al aparecer el *Quijote* de 1615 en calidad de *Segunda parte* de la obra, quedó revocada de hecho la partición en cuatro secciones del volumen publicado diez años antes por Cervantes.²

Es la primera obra genuinamente desmitificadora de la tradición **caballeresca** y **cortés** por su tratamiento burlesco. Representa la primera **novela** moderna y la primera **novela polifónica**; como tal, ejerció un enorme influjo en toda la **narrativa europea**. Por considerarse «el mejor trabajo literario jamás escrito», encabezó la **lista de las mejores obras literarias de la historia**, que se estableció con las votaciones de cien grandes escritores de 54 nacionalidades a petición del Club Noruego del Libro y Bokklubben World Library en 2002; así, fue la única excepción en el estricto orden alfabético que se había dispuesto.³

El ingenioso hidalgo don Quixote de la Mancha (1605) Segunda parte del ingenioso caballero don Quixote de la Mancha (1615)

de **Miguel de Cervantes Saavedra**

EL INGENIOSO HIDALGO DON QUIXOTE DE LA MANCHA.

Compuesto por Miguel de Cervantes Saavedra.

DIRIGIDO AL DUQUE DE BEJAR;
Marques de Gibralfaro, Conde de Benalcazar, y Bañares,
Vizconde de la Puebla de Alcocer, Señor de las villas de Capilla, Curiel, y Burgoillos.



Año,

1605.



Incluso los LLMs más básicos tienen mucha información en poco espacio



Parámetros
~140 GB



Código
**~500
líneas¹**

Fuente: **Andrej Karpathy**

<https://github.com/tairov/llama2.py/blob/master/llama2.py>



Como se entrena un LLM



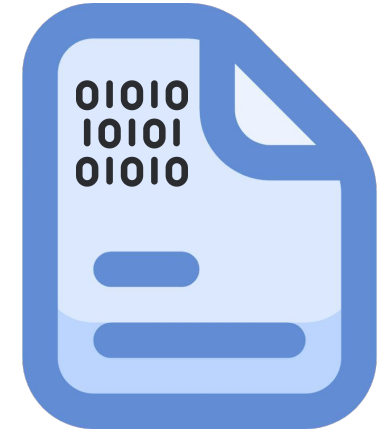
Entrenar los LLMs es algo más complejo



Pedazo de internet
~**10 GB** de
texto



6K GPUS por 12 días, ~2M\$
~**1e24 Flops**



Parámetros
~**140 GB**



¿Cómo se entrena un LLM?



<USER>

Can you help me with this code? It seems like there is a bug.
`print("hello world")`

<ASSISTANT>

It looks like you forgot to close the string passed to the function print. You have to add a closing quote to properly terminate the string. Here is the corrected function:

```
print("hello world")
```

Let me know if I can help with anything else!



Anualmente



Fase 1: Preentrenamiento

1. Descarga ~10TB de texto
2. Obtén un clúster de 6K GPUS
3. Comprime el texto en una red neuronal, paga 2M\$ y espera 12 días
4. Obtén un **modelo base**

Semanalmente



Fase 2: Ajuste fino

1. Define política de etiquetado
2. Contrata gente, y obten 100K preguntas Q&A o comparaciones
3. Haz ajuste fino del modelo, espera 1 día
4. Obtén un **modelo asistente**
5. Realiza un montón de pruebas
6. Despliega
7. Monitoriza, y obten errores, vuelve al 1

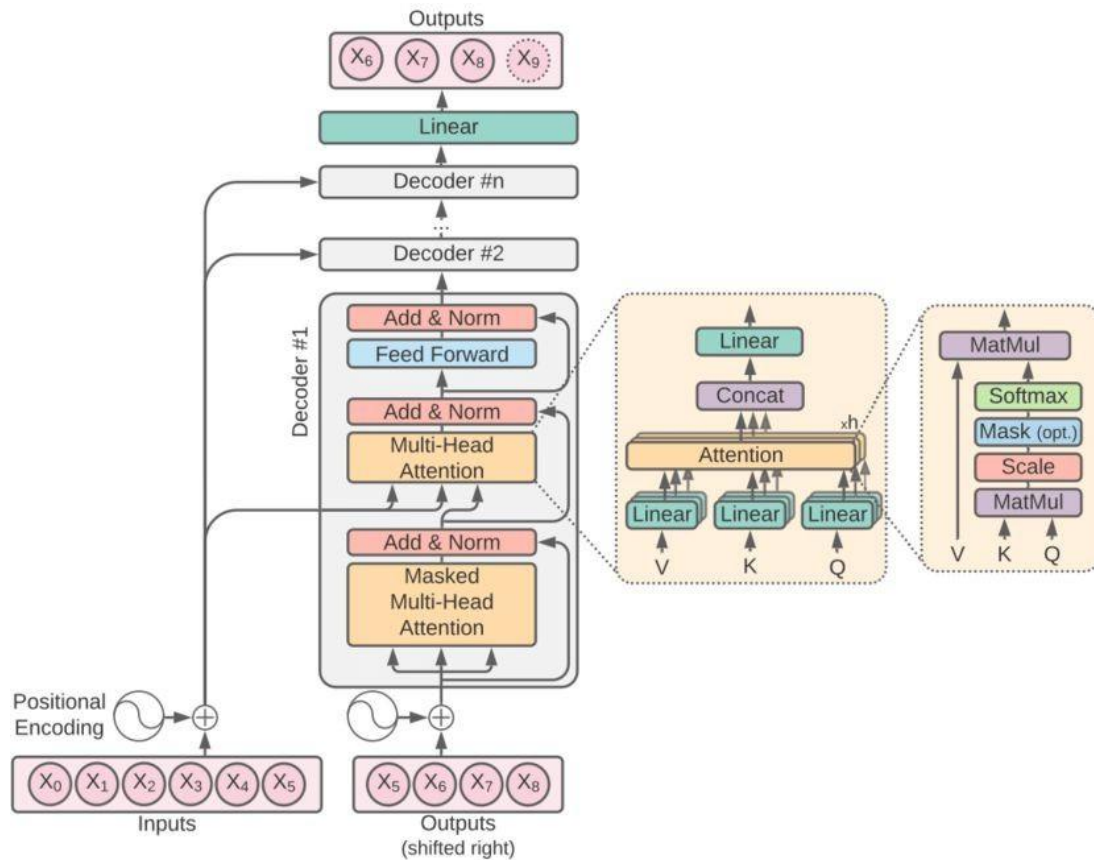


Model	Training end	Chip type	TFLOP/s (max)	Chip count	Wall clock (days)	Total time (years)	Retail (US\$)	MMLU
GPT-3 175B	Apr/2020	V100	130	10,000	15 days	405y	\$9M	43.9
Llama 1 65B	Jan/2023	A100	312	2,048	21 days	118y	\$4M	63.4
Llama 2 70B	Jun/2023	A100	312	2,048	35 days	196y	\$7M	68.0
Titan 200B	Apr/2023	A100	312	13,760	48 days	1,319y	\$45M	70.4
GPT-4 1.7T	Aug/2022	A100	312	25,000	95 days	6,507y	\$224M	86.4
Gemini	Nov/2023	TPUv4	275	57,000	100 days	15,616y	\$440M	90.0
Llama 3 405B	Apr/2024	H100	989	24,576	50 days	3,366y	\$125M	85+
GPT-5	Apr/2024	H100	989	50,000	120 days	16,438y	\$612M	
Grok 2	Jun/2024	H100	989	20,000	50 days	6,571y	\$245M	
Olympus	Aug/2024	H100	989					
Gemini 2	Nov/2024	TPUv6	1,847					
Grok 3	Dec/2024	H100	989	100,000	50 days	32,855y	\$1.2B	
Alan D. Thompson. May/2024. LifeArchitect.ai								

Table. Model training compute (see working, with sources⁸).



Proceso de entrenamiento



★ Los LLMs son parcialmente cajas negras

Aprendizaje autosupervisado

- La red aprende a predecir el siguiente token cada vez mejor.
- Podemos medir si funciona bien, pero no interpretar fácilmente cómo millones/billones de parámetros producen cada respuesta.

¿Quién es la madre de Tom Cruise?

Mary Lee Pfeiffer ✓

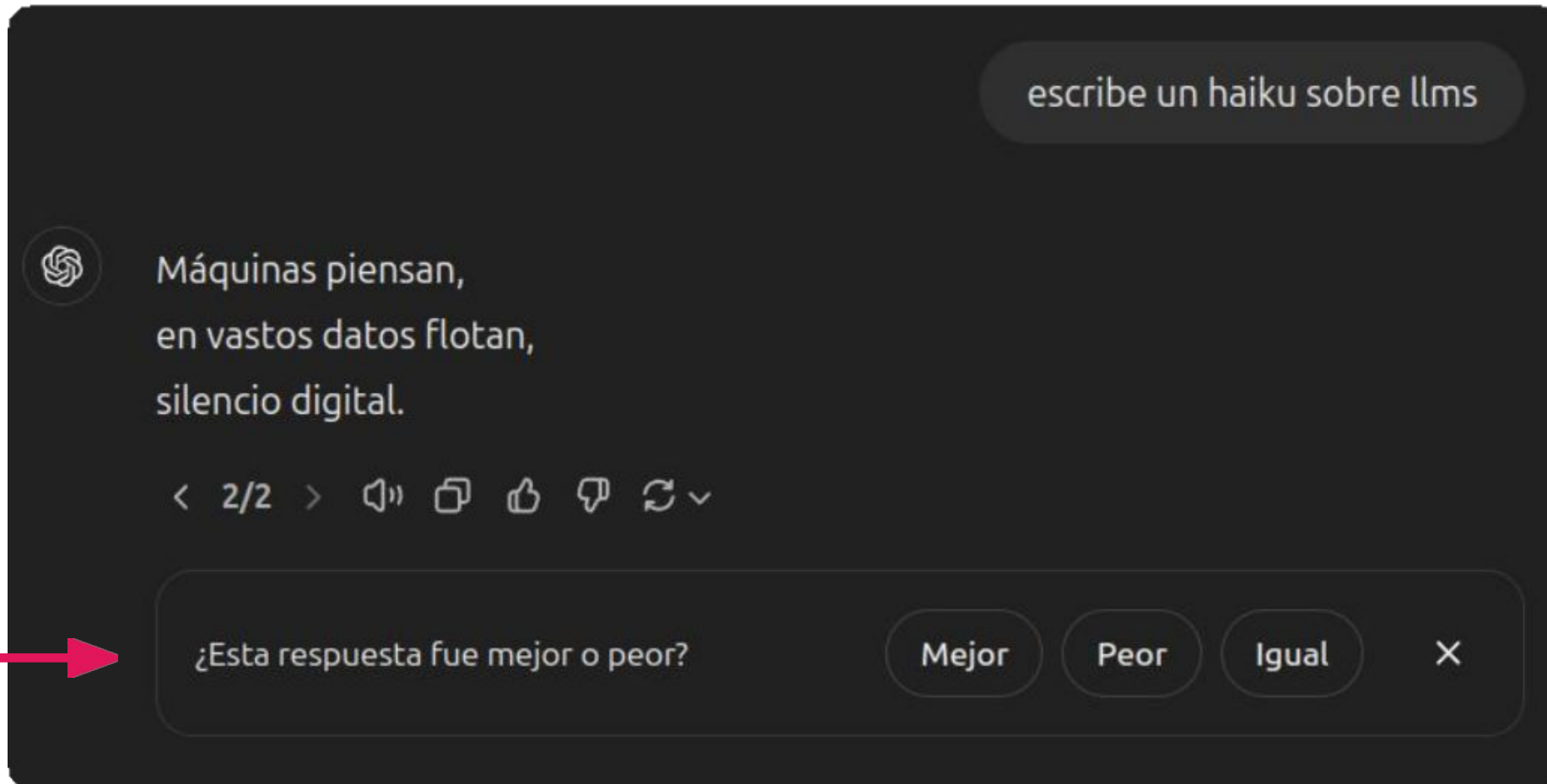
¿Quien es el hijo de Mary Lee Pfeiffer?

Puede fallar ✗



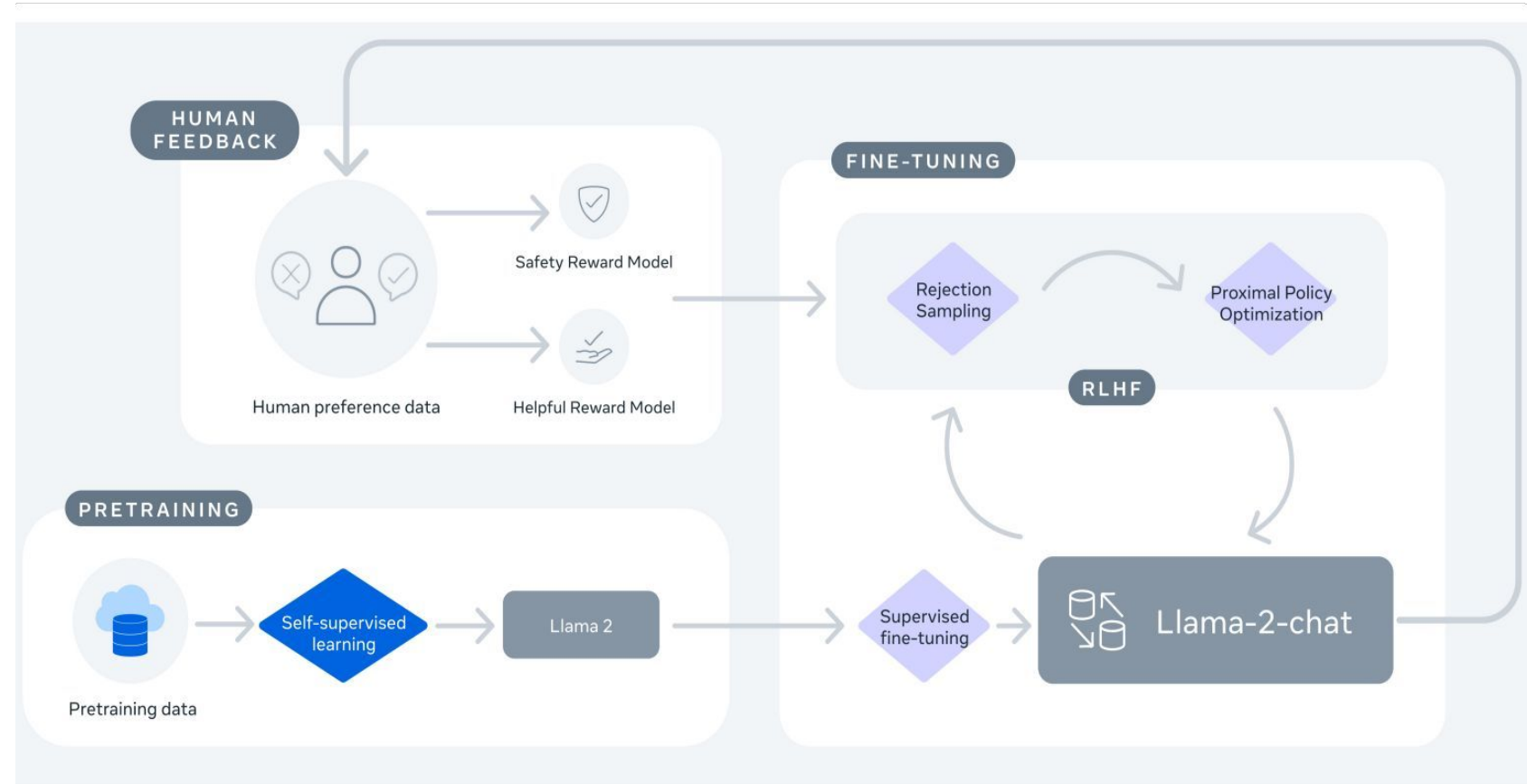
Un LLM puede acertar una pregunta memorizada y fallar al transformar la misma información en otra forma lógica.

Comparaciones para mejorar modelos



Proceso entrenamiento de un LLM

- **Pretraining:** crea el modelo base.
- **Fine-tuning:** le enseña a seguir instrucciones.
- **Feedback humano:** introduce preferencias de calidad y seguridad.
- **RLHF:** optimiza el comportamiento del asistente.
- **Iteración:** se evalúa, corrige y mejora continuamente.

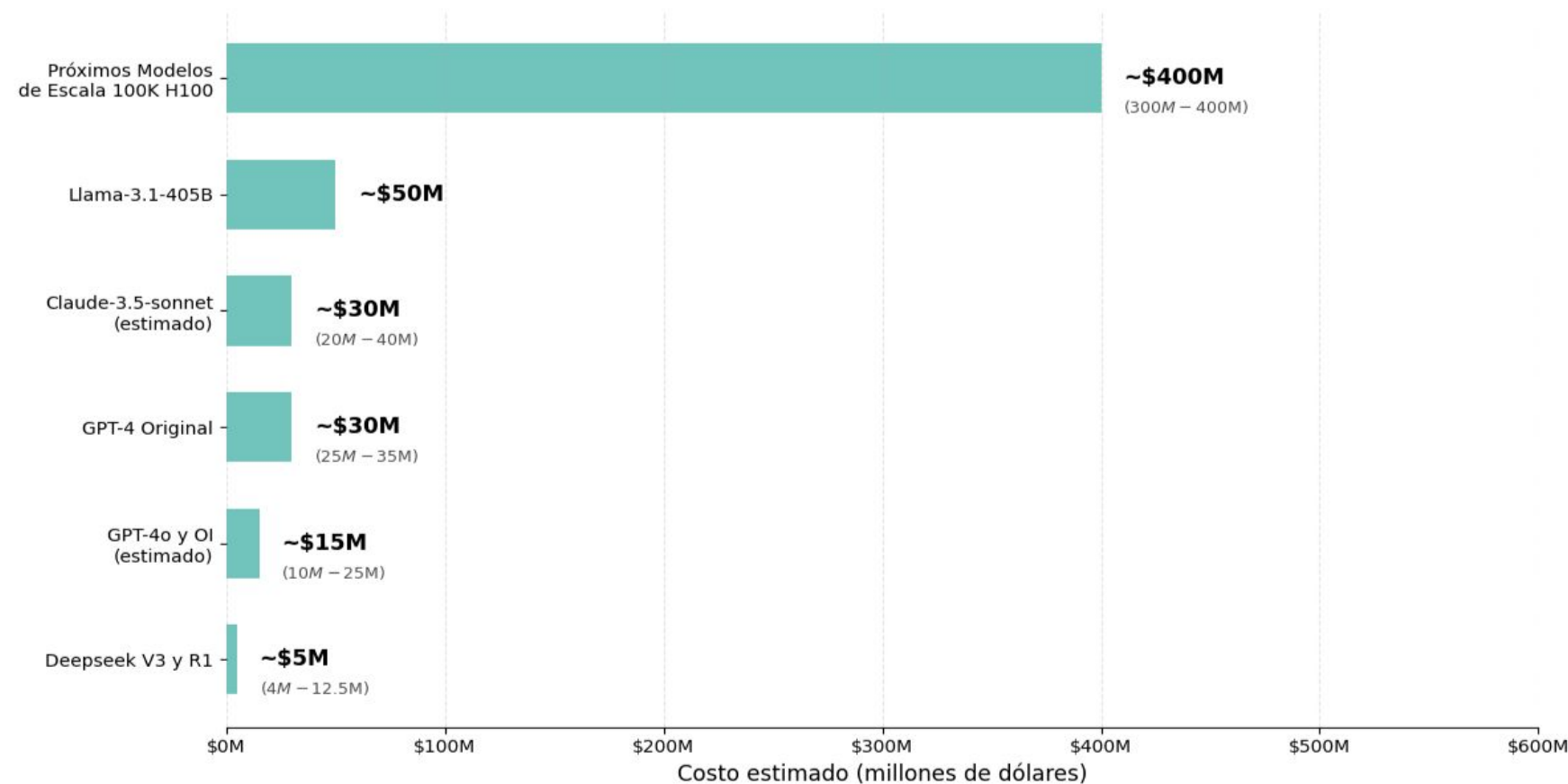


¿Cuánto cuesta entrenar un modelo?



Costo estimado de varios modelos usando precios de cómputo de 2025, en H100 horas de cómputo de entrenamiento

Comparación de Costos de Entrenamiento de Modelos de IA



(Los rangos indican intervalos de confianza del 85%)



¿Cuánto cuesta entrenar un modelo?

- **GPT-3**
 - Coste estimado de entrenamiento: ~4,6M\$
 - Estimación basada en coste de cómputo cloud; no es una cifra oficial cerrada de OpenAI.
- **GPT-4**
 - Coste reportado: >100M\$
 - Sam Altman (CEO OpenAI) confirmó que el entrenamiento de GPT-4 superó los 100 millones de dólares.
- **¿Dónde está el coste?**
 - GPUs / aceleradores especializados.
 - Tiempo de entrenamiento y experimentación.
 - Infraestructura de centros de datos.
 - Personal investigador e ingeniería.
 - Datos, limpieza, evaluación y seguridad.
- **Tendencia 2026**
 - Los mejores modelos se están moviendo hacia entrenamientos de cientos o incluso miles de millones de dólares, incluyendo cómputo, infraestructura, talento y evaluación, pero ya es complicado saber cuanto porque no están publicando los datos.



Uso de un LLM



Qué recibe el modelo

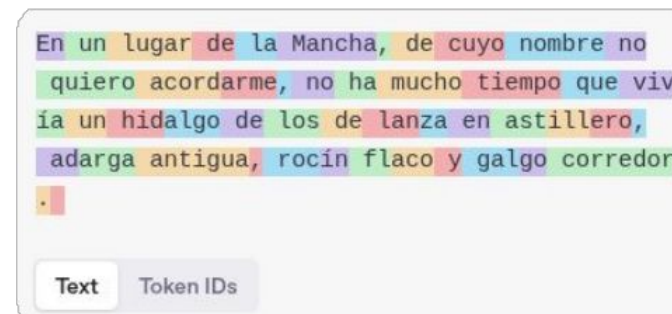
El modelo no responde solo a tu última pregunta

- **Instrucciones del sistema**
 - Reglas generales: comportamiento, seguridad, tono y límites.
- **Prompt del usuario**
 - La petición concreta que escribimos.
- **Historial de conversación**
 - Mensajes anteriores que siguen siendo relevantes.
- **Documentos o contexto añadido**
 - PDFs, tablas, fragmentos RAG, texto pegado o datos externos.
- **Herramientas disponibles**
 - Buscador, código, calendario, base de datos, APIs, etc.
- **Parámetros de generación**
 - Modelo, temperatura, longitud máxima, streaming, etc.



Ventana de contexto

- **Cantidad máxima** de información que el modelo puede tener en cuenta a la vez
- Se mide en tokens, no en páginas ni palabras
- Un token puede ser una palabra corta, parte de una palabra o un signo de puntuación
- La ventana incluye:
 - Instrucciones del sistema
 - El prompt
 - Documentos o contexto añadido
 - Historial relevante de la conversación
 - Respuesta generada por la IA
- Si se supera el límite, parte del contexto puede recortarse, resumirse o perderse.
- Cómo trabajar con documentos largos
 - **Compactar:** resumir periódicamente la información importante (**Importantísimo en agentes**)
 - **Segmentar:** dividir documentos o tareas grandes en partes
 - **Recuperar solo lo relevante:** usar búsqueda/RAG en vez de cargar todo
 - **Optimizar prompts:** instrucciones claras, concisas y específicas
 - **Priorizar información crítica:** separar objetivo, datos e instrucciones



Cómo genera la respuesta

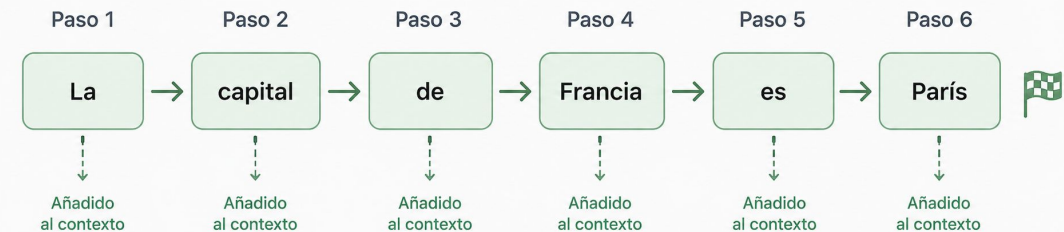
Token a token

- El modelo no genera toda la respuesta de golpe.
- En cada paso predice el siguiente token más probable según el contexto.
- Cada token generado se añade al contexto y condiciona el siguiente.
- El proceso continúa hasta que:
 - llega a un token de fin
 - alcanza el límite de longitud
 - se corta por configuración
- Por eso las respuestas largas consumen más:
 - tokens
 - tiempo
 - coste

Ejemplo simple:

Prompt: "La capital de Francia es..."

Generación:



Parámetros de generación

Por qué una misma pregunta puede dar respuestas distintas

- Max tokens
 - Limita la longitud máxima de la respuesta.
- Temperature
 - Controla la variabilidad:
 - baja → más estable y conservador
 - alta → más creativo, pero menos predecible
- Top-p
 - Limita el conjunto de tokens candidatos según probabilidad acumulada.
- Stop sequences
 - Indican al modelo cuándo debe parar.
- Seed
 - Ayuda a reproducir resultados cuando el proveedor lo permite.
- Streaming
 - Muestra la respuesta mientras se genera, en lugar de esperar al final.

```
from openai import OpenAI

client = OpenAI()

response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "system", "content": "Eres un asistente útil."},
        {"role": "user", "content": "Explicame la fotosíntesis en pocas palabras."}
    ],
    max_tokens=300,
    temperature=0.7,
    top_p=0.9,
    stop=["\n\n", "###"],
    seed=42,
    stream=True
)
```

MAX TOKENS
Limita la longitud máxima de la respuesta.

TEMPERATURE
Controla la variabilidad.
Baja → más estable.
Alta → más creativo.

TOP-P
Limita el conjunto de tokens candidatos según probabilidad acumulada.

STOP SEQUENCES
Indican al modelo cuándo debe parar.

SEED
Ayuda a reproducir resultados cuando el proveedor lo permite.

STREAMING
Muestra la respuesta mientras se genera, en lugar de esperar al final.

Ejemplo: temperature baja (0.2)

La fotosíntesis es el proceso por el cual las plantas convierten la luz solar en energía. Usan dióxido de carbono y agua para producir glucosa y oxígeno.

Ejemplo: temperature alta (1.0)

La fotosíntesis es como el superpoder de las plantas. Con la luz del sol, crean su propia comida y liberan oxígeno al aire, ayudando a la vida en la Tierra.

```
# Ejecutar y recibir la respuesta (streaming)
for chunk in response:
    print(chunk.choices[0].delta.content, end="", flush=True)
```

Prompt Engineering

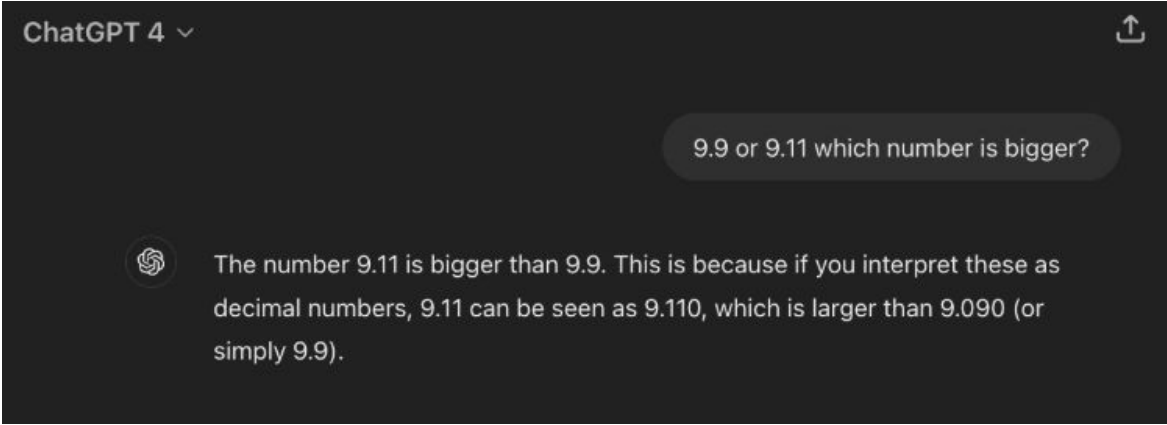
- **Dar contexto**
 - Qué problema estamos resolviendo y para quién.
- **Definir el objetivo**
 - Qué queremos obtener exactamente.
- **Asignar un rol si ayuda**
 - “Actúa como analista de datos”, “como profesor”, “como revisor técnico...”
- **Especificar el formato de salida**
 - Tabla, bullets, JSON, email, resumen ejecutivo, pasos, etc.
- **Añadir restricciones**
 - Longitud, tono, idioma, fuentes, supuestos, nivel técnico.
- **Incluir ejemplos**
 - Especialmente si queremos un estilo o formato concreto.
- **Iterar**
 - El primer prompt rara vez es el definitivo.

Plantilla útil

Actúa como [rol].
Necesito [objetivo].
Contexto: [datos].
Devuelve la respuesta en [formato].
Ten en cuenta [restricciones].

Un LLM también puede fallar

Riesgo	Cómo reducirlo
Alucinaciones: información convincente pero falsa	Pedir fuentes o evidencias
Instrucciones ambiguas	Separar objetivo, contexto y formato
Errores en el contexto	Revisar los datos de entrada
Fechas o fuentes mezcladas	Verificar información actualizada
Cálculos o razonamientos largos	Usar herramientas externas
Respuesta sin suficiente información	Pedir que indique incertidumbre
Forma correcta, contenido incorrecto	Validar antes de tomar decisiones



Finetuning



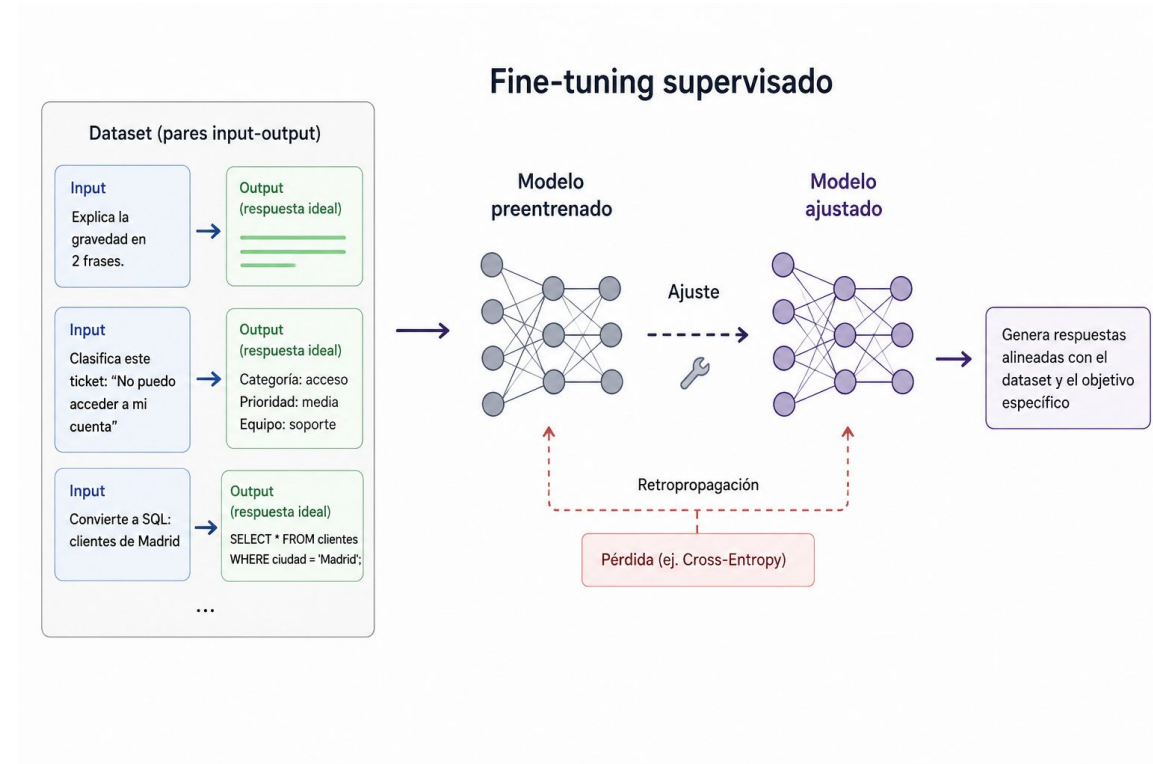
¿Qué es el fine-tuning?

- El fine-tuning consiste en **adaptar un modelo preentrenado a una tarea**, dominio o caso de uso específico.
- Parte del conocimiento general aprendido durante el pretraining y continúa entrenando el modelo con un dataset más pequeño y especializado.
- Es una forma de **transfer learning**: reutilizamos lo que el modelo ya sabe y lo ajustamos a una necesidad concreta.
- Históricamente se ha usado para especializar modelos en dominios muy concretos: medicina, legal, finanzas, código o visión artificial.
- Ejemplos:
 - Medicina: clasificación de informes clínicos o extracción de información de historiales.
 - Legal: análisis de contratos, cláusulas y jurisprudencia.
 - Finanzas: análisis de riesgo, reporting regulatorio o clasificación de operaciones.
 - Código: modelos adaptados a programación, frameworks o patrones concretos.
 - Marca/soporte: tono corporativo, estructura de respuesta y clasificación de tickets.
- En la práctica, no suele ser la primera opción: requiere datos de calidad, evaluación y coste de entrenamiento. Normalmente se prueba antes con prompting o RAG, y se usa fine-tuning cuando el patrón es estable y se repite mucho.



Fine-tuning supervisado

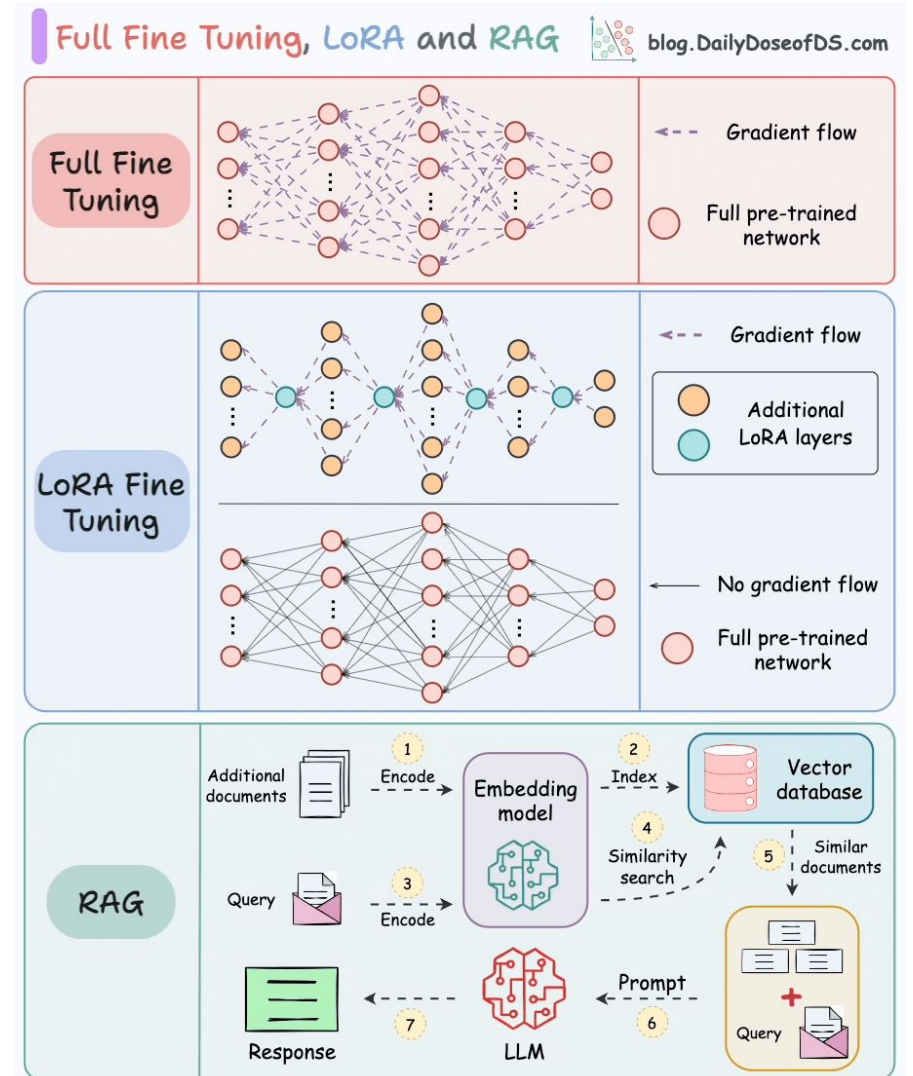
- Partimos de un modelo preentrenado que ya sabe lenguaje general, pero no necesariamente cómo responder en nuestro caso concreto.
- Creamos un dataset con muchos ejemplos de entrada y respuesta ideal.
- En cada ejemplo, el modelo intenta generar la respuesta esperada.
- Se compara la respuesta del modelo con la respuesta ideal y se calcula el error.
- Mediante backpropagation, se ajustan algunos pesos del modelo para reducir ese error.
- Tras repetir este proceso con muchos ejemplos, el modelo aprende el patrón de respuesta deseado.
- El modelo aprende a responder siguiendo ese formato, criterio y estilo de forma consistente.



LoRA, riesgos y evaluación

- Fine-tunear todos los parámetros de un LLM puede ser muy caro.
- **LoRA** reduce el coste entrenando pequeñas capas adicionales sobre el modelo base, manteniendo congelada la mayoría de pesos.
- **Ventajas:**
 - Menor coste de entrenamiento.
 - Menos requisitos de hardware.
 - Posibilidad de crear adaptadores por tarea o dominio.
- **Riesgos:**
 - Sobreajuste.
 - Datos sensibles.
 - Sesgos.
 - Pérdida de capacidades generales.
 - Respuestas demasiado rígidas.

Antes de usarlo en producción hay que evaluar calidad, seguridad, formato, coste, latencia y casos límite.



¿Cuándo usar cada método?

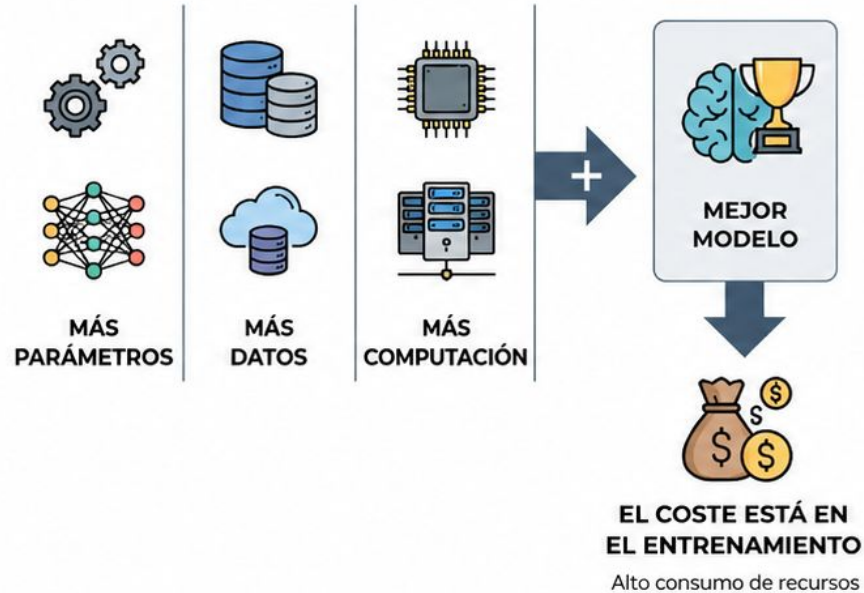


Razonamiento en Inferencia



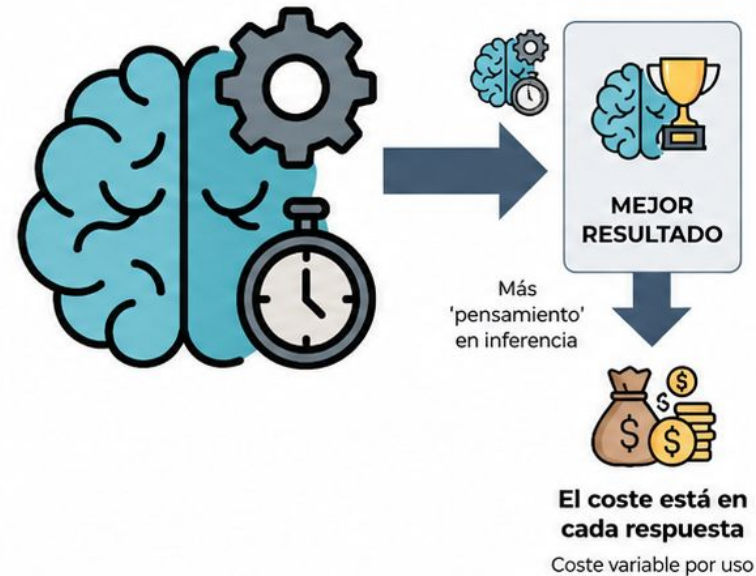
Test-Time Compute: pensar más antes de responder

Antes (Scaling laws tradicional)



- La mejora venía de modelos cada vez más grandes.
- Mucho cómputo, GPUs y tiempo durante el entrenamiento.
- Caro de entrenar, más barato de usar.

Ahora (Test-Time Compute)



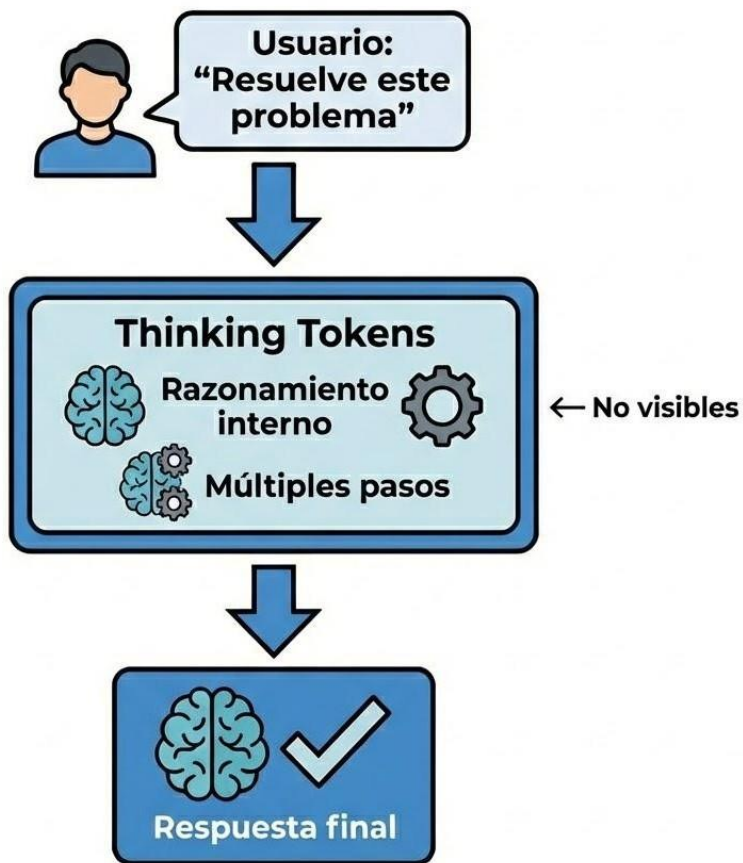
- El modelo dedica más tiempo y cómputo a “pensar”.
- Puede explorar, verificar, descomponer y corregir.
- Cada respuesta cuesta más, pero el modelo base puede ser más pequeño.



Idea clave: Antes ganábamos tamaño en entrenamiento. Ahora también ganamos “pensamiento” en inferencia. Más computación no solo para entrenar modelos más grandes, sino para obtener mejores respuestas.



Test-Time Compute: pensar más antes de responder



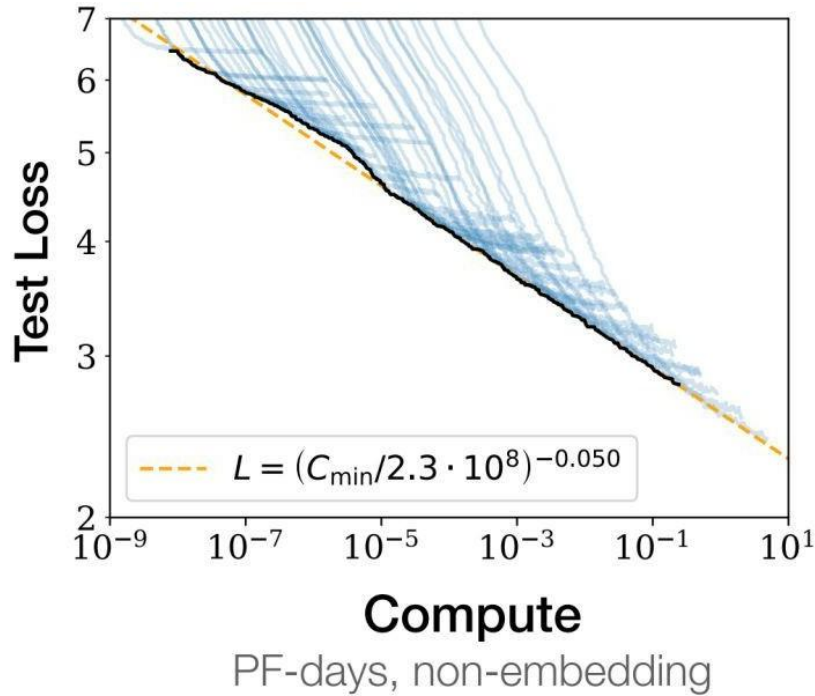
El modelo genera **tokens de pensamiento** antes de responder.

Más tokens = más razonamiento = mejor resultado (para problemas difíciles).

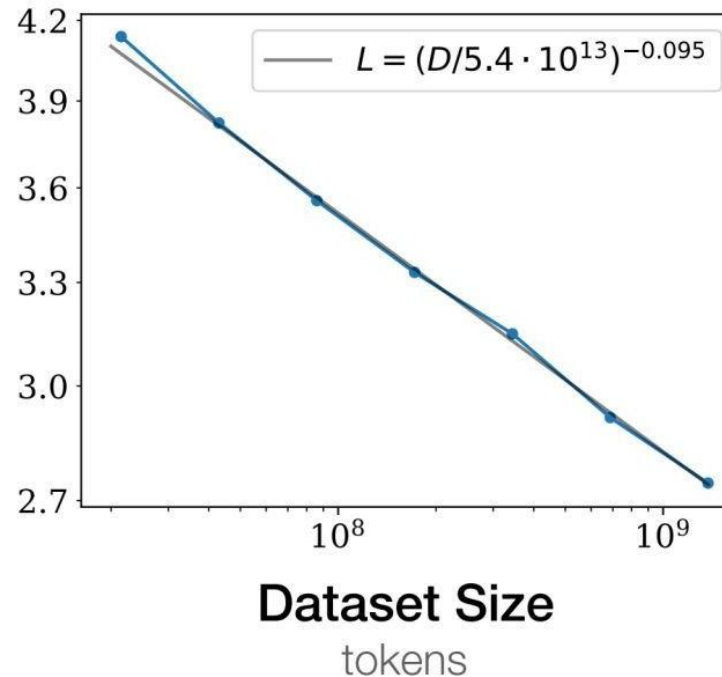
Tarea	GPT-4o	o1
MATH	60%	94%
GPQA Diamond	50%	78%
Codeforces	11%	89%
PhD-level science	~60%	~80%

Leyes de escalado

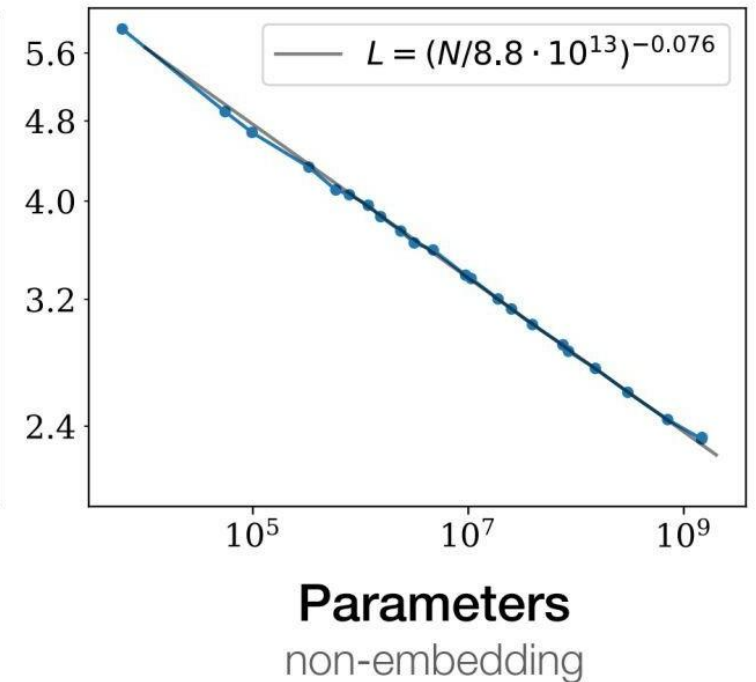
Más computo → menor error



Más datos → menor error



Más parámetros → menor error



★ Más escala suele producir modelos más capaces



¿Cuándo usar modelos de razonamiento?

Hace 1 año, se podía elegir si coger razonamiento o no. Hoy casi todos los modelos razonan algo, porque se ha visto que es lo que mejor funciona. La decisión práctica es cuánto coste, latencia y profundidad merece la tarea.

Más razonamiento

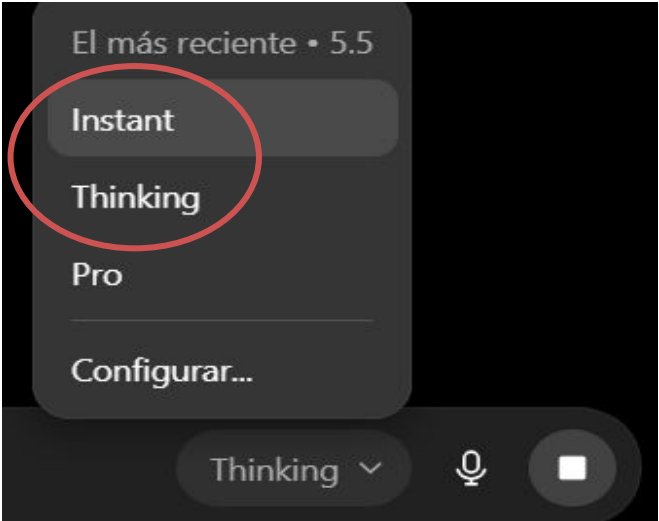
- **Código complejo:** arquitectura, bugs difíciles, refactorings grandes.
- **Matemáticas / ciencia:** problemas no triviales o con verificación.
- **Legal / compliance:** contratos, regulación, riesgos.
- **Planificación:** estrategia, decisiones multi-paso, trade-offs.
- **Análisis ambiguo:** comparar opciones, detectar supuestos, justificar.

Razonamiento ligero

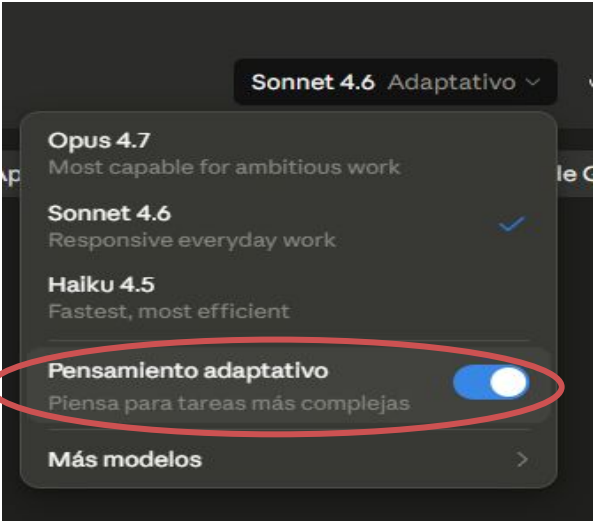
- **Chat general:** preguntas rápidas o conversación.
- **Resumen simple:** extraer ideas principales.
- **Traducción / reformulación:** cambiar idioma, tono o formato.
- **Extracción simple:** clasificar, etiquetar, sacar campos.
- **Alto volumen:** muchas llamadas donde importan coste y velocidad.



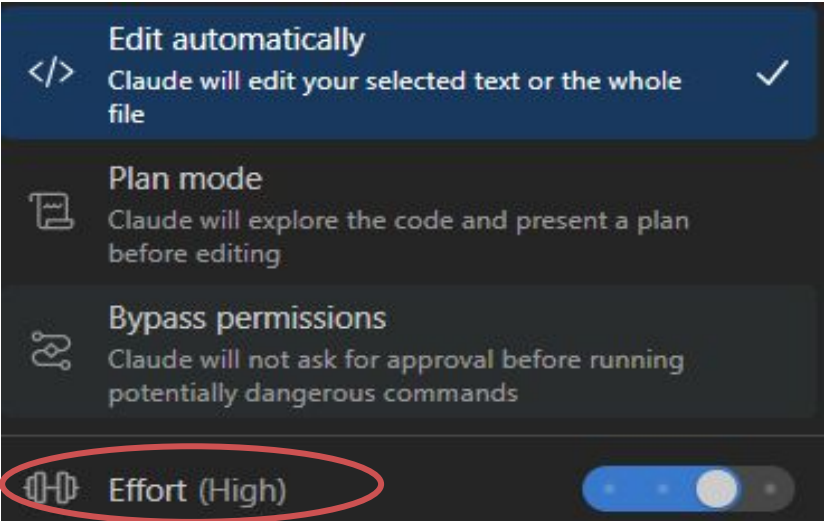
¿Cuándo usar modelos de razonamiento?



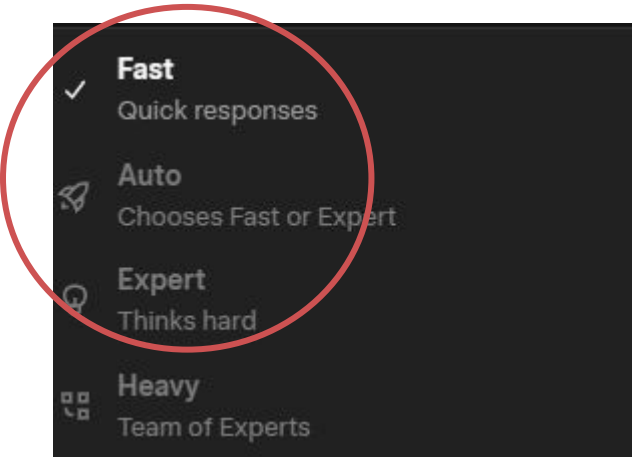
Chatgpt



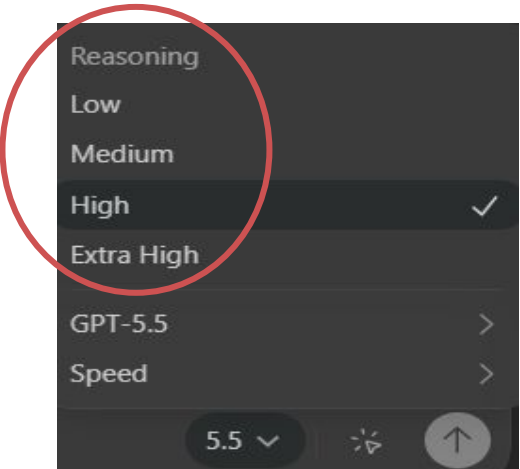
Claude



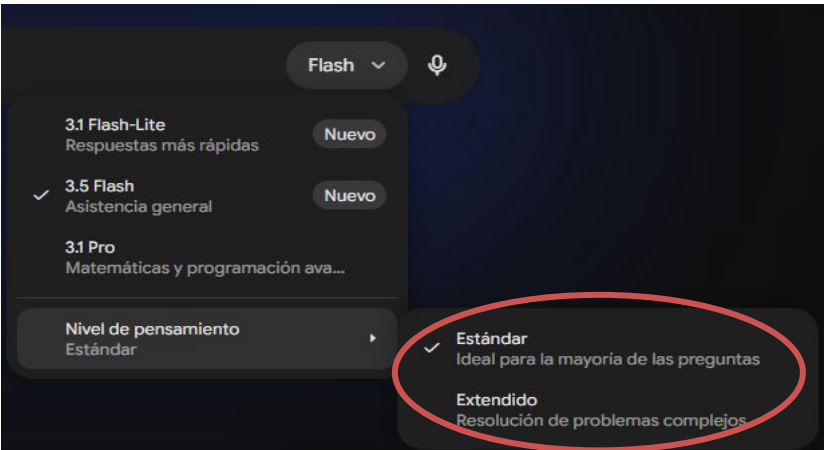
Claude Code



Grok



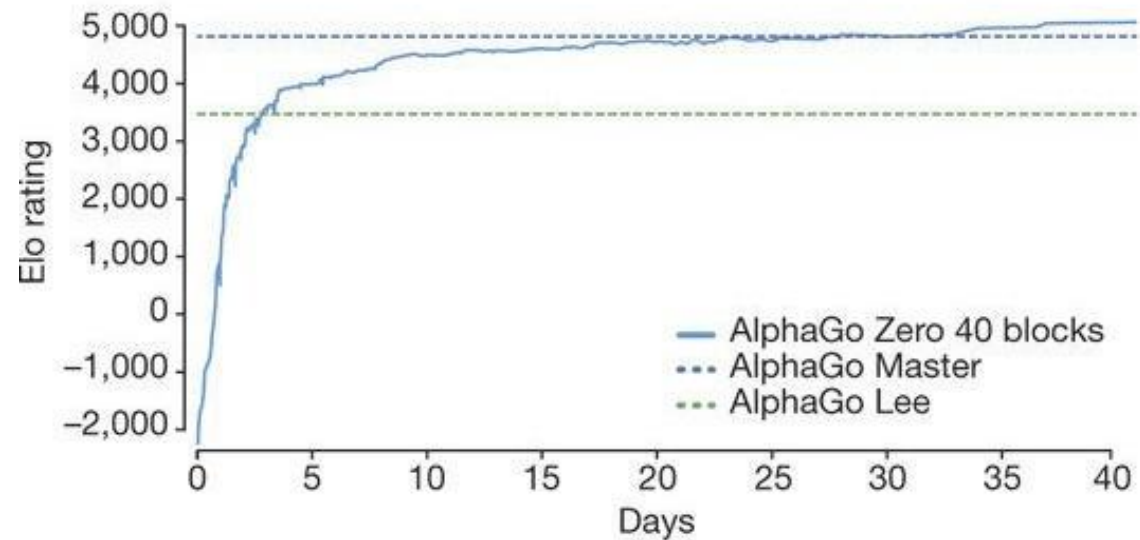
Codex



Gemini

La siguiente frontera: Autoaprendizaje

AlphaGo aprendía jugando contra sí mismo porque tenía una recompensa clara: ganar o perder. En LLMs el reto es que muchas tareas abiertas no tienen una recompensa tan objetiva.



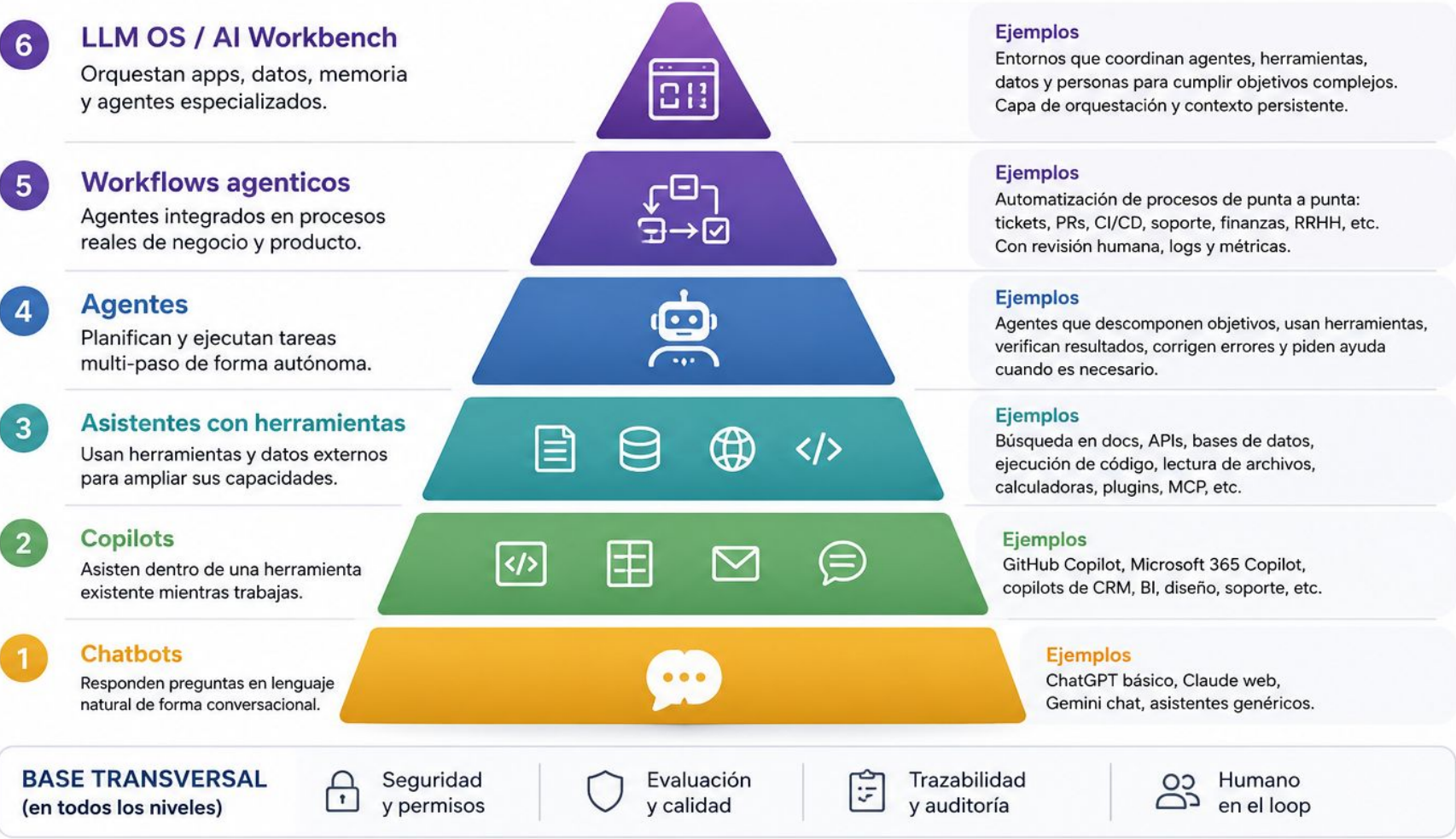
AlphaGo: supervisado + refuerzo + autojuego

1. Imitación: aprende de humanos.
2. Autojuego: mejora jugando contra sí mismo.
3. Recompensa clara: ganar o perder.

¿Cómo aplicarlo en LLMs?

En código, matemáticas o juegos podemos verificar el resultado. En problemas abiertos no siempre hay una respuesta única. Desafío: Definir buenas recompensas para que el modelo mejore sin degradar calidad, seguridad o criterio.

De chatbots a sistemas agénticos



Benchmark LLMs



¿Cómo comparamos LLMs?

- No existe un único ranking definitivo
- Cada benchmark mide una capacidad concreta:
 - razonamiento
 - matemáticas
 - código
 - conocimiento general
 - seguimiento de instrucciones
 - multimodalidad
- Un modelo puede ser muy bueno en una tarea y mediocre en otra
- Además de calidad, hay que mirar:
 - coste
 - latencia
 - ventana de contexto
 - fiabilidad
 - privacidad / despliegue
- No existe “el mejor modelo”, existe el mejor modelo para cada caso de uso

	Claude 3.5 Sonnet	Claude 3 Opus	GPT-4o	Gemini 1.5 Pro	Llama-400b (early snapshot)
Graduate level reasoning <i>GPQA, Diamond</i>	59.4%* 0-shot CoT	50.4% 0-shot CoT	53.6% 0-shot CoT	—	—
Undergraduate level knowledge <i>MMLU</i>	88.7%** 5-shot	86.8% 5-shot	—	85.9% 5-shot	86.1% 5-shot
	88.3% 0-shot CoT	85.7% 0-shot CoT	88.7% 0-shot CoT	—	—
Code <i>HumanEval</i>	92.0% 0-shot	84.9% 0-shot	90.2% 0-shot	84.1% 0-shot	84.1% 0-shot
Multilingual math <i>MGSM</i>	91.6% 0-shot CoT	90.7% 0-shot CoT	90.5% 0-shot CoT	87.5% 8-shot	—
Reasoning over text <i>DROP, F1 score</i>	87.1 3-shot	83.1 3-shot	83.4 3-shot	74.9 Variable shots	83.5 3-shot Pre-trained model
Mixed evaluations <i>BIG-Bench-Hard</i>	93.1% 3-shot CoT	86.8% 3-shot CoT	—	89.2% 3-shot CoT	85.3% 3-shot CoT Pre-trained model
Math problem-solving <i>MATH</i>	71.1% 0-shot CoT	60.1% 0-shot CoT	76.6% 0-shot CoT	67.7% 4-shot	57.8% 4-shot CoT
Grade school math <i>GSM8K</i>	96.4% 0-shot CoT	95.0% 0-shot CoT	—	90.8% 11-shot	94.1% 8-shot CoT

* Claude 3.5 Sonnet scores 67.2% on 5-shot CoT GPQA with maj@32

** Claude 3.5 Sonnet scores 90.4% on MMLU with 5-shot CoT prompting



¿Qué miden los benchmarks?

- **MMLU**
 - Conocimiento general en muchas materias: ciencia, historia, derecho, medicina, etc.
- **GPQA Diamond**
 - Razonamiento científico difícil, nivel posgrado.
- **HumanEval**
 - Generación de código Python a partir de enunciados.
- **MATH / GSM8K**
 - Problemas matemáticos y razonamiento paso a paso.
- **DROP**
 - Comprensión lectora con razonamiento sobre textos.
- **BBH / BIG-Bench Hard**
 - Tareas difíciles de razonamiento, lógica e instrucciones.
- **SWE-bench / LiveCodeBench**
 - Programación más realista: bugs, repositorios, issues y tests.
- **Chatbot Arena / LMArena**
 - Preferencia humana: dos respuestas anónimas y el usuario elige la mejor.

Cada poco salen nuevos Benchmarks porque los modelos mejoran y también porque suelen “hacer trampas” entrenándose con las soluciones a estos benchmarks



Evaluación de LLMs

- Compara modelos usando preferencias humanas
- Dos modelos responden a la misma pregunta de forma anónima
- El usuario elige qué respuesta prefiere
- Con muchas comparaciones se construye un ranking tipo Elo
- Es útil para medir:
 - claridad
 - utilidad
 - estilo
 - seguimiento de instrucciones
 - experiencia percibida
- La idea es evaluar modelos por preferencia de los humanos más que una nota en “benchmarks”

Text Arena Overall

View overall rankings across various AI models in text-to-text tasks across math, coding, creative writing, and other open-ended domains.

🕒 May 17, 2026 🗳️ 6,297,180 votes 📄 360 models

⚙️ Hide Filters Rank by Models Labs 🔍 🗑️

Rank	Rank Spread	Model	Score	Votes	Price \$/M	Context
1	1 → 4	claude-opus-4-6-thinking Anthropic · Proprietary	1502 ±4	27,454	\$5 / \$25	1M
2	1 → 5	claude-opus-4-7-thinking Anthropic · Proprietary	1500 ±6	12,920	\$5 / \$25	1M
3	1 → 5	claude-opus-4-6 Anthropic · Proprietary	1498 ±4	29,240	\$5 / \$25	1M
4	1 → 9	claude-opus-4-7 Anthropic · Proprietary	1492 ±6	13,571	\$5 / \$25	1M
5	2 → 12	muse-spark Meta · Proprietary	1489 ±6 Preliminary	11,103	N/A	N/A
6	4 → 12	gemini-3.1-pro-preview Google · Proprietary	1488 ±4	34,189	\$2 / \$12	1M
7	4 → 13	gemini-3-pro Google · Proprietary	1486 ±4	41,331	\$2 / \$12	1M
8	4 → 21	gpt-5.5-high OpenAI · Proprietary	1481 ±6	10,172	\$5 / \$30	1.1M
9	4 → 22	gemini-3.5-flash Google · Proprietary	1480 ±8 Preliminary	5907	\$1.50 / \$9	1M

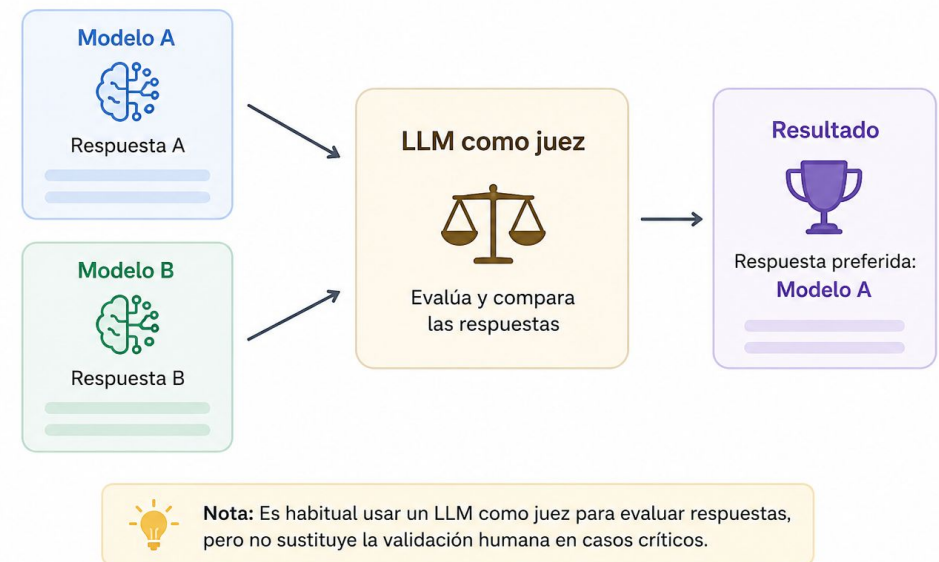
<https://lmarena.ai/>



Siempre hacer evaluación propia!

- Usar benchmarks públicos para hacer una primera shortlist
- Probar los modelos con ejemplos reales de tu caso de uso
- Medir:
 - calidad de respuesta
 - errores críticos
 - coste por tarea
 - latencia
 - estabilidad
 - facilidad de integración
- Crear un pequeño dataset interno de evaluación
 - Repetir pruebas cuando cambie el modelo, el prompt o el proveedor
- Existen librerías y proveedores para hacer evaluaciones de manera fácil con los LLMs.
 - Proveedores: Langfuse, arizephoenix...
 - Librerías: DeepEval, Ragas...

Evaluación con LLM como juez



¿Qué se suele evaluar propia con un LLM?

- **Calidad de respuesta**
 - Si la respuesta es útil, completa y bien estructurada.
- **Exactitud factual**
 - Si los datos, fechas, nombres y afirmaciones son correctos.
- **Seguimiento de instrucciones**
 - Si respeta el formato, el tono, el idioma y las restricciones pedidas.
- **Razonamiento**
 - Si resuelve tareas complejas, compara opciones y justifica bien.
- **Robustez**
 - Si funciona bien con casos ambiguos, incompletos o difíciles.
- **Trustworthiness**
 - Si reconoce incertidumbre, no inventa y usa fuentes cuando hacen falta.
- **Errores críticos**
 - Si evita respuestas peligrosas, sesgadas o inaceptables para el caso de uso.
- **Coste y latencia**
 - Cuánto cuesta cada tarea y cuánto tarda en responder.
- **Estabilidad**
 - Si responde de forma consistente ante prompts similares.
- **Integración**
 - Si encaja bien con APIs, herramientas, datos internos y flujos de trabajo.

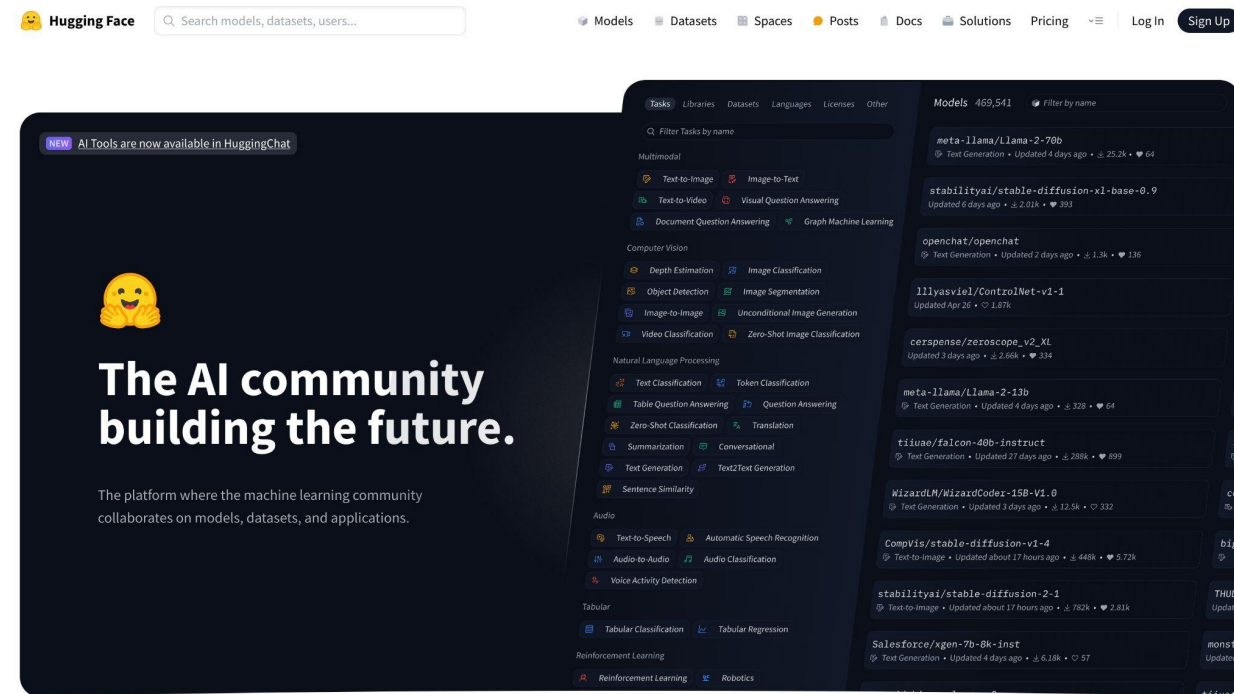


Explotación de modelos con Hugging Face



¿Qué es Hugging Face?

- La plataforma de referencia para compartir y usar modelos de IA open-source.
 - Model Hub: modelos preentrenados listos para usar: LLMs, embeddings, visión, audio, NLP, etc.
 - Datasets: repositorio de corpus y datasets para entrenamiento, evaluación y pruebas.
 - Transformers: librería principal para cargar, entrenar y desplegar modelos.
 - Spaces: demos y aplicaciones interactivas para probar modelos en navegador.
 - Comunidad: investigadores, empresas y desarrolladores publican modelos, benchmarks y recursos.
- Recursos:
 - Plataforma: <https://huggingface.co/>
 - Curso introductorio: <https://huggingface.co/learn/llm-course/chapter1/1>



Hugging Face para buscar modelos (Legacy)

The screenshot displays the Hugging Face website interface. At the top, there is a navigation bar with links for Models, Datasets, Spaces, Docs, Solutions, and Pricing. A search bar is located next to the Hugging Face logo. Below the navigation bar, a banner message reads "Hugging Face is way more fun with friends and colleagues!".

On the left side, there are several filter categories:

- Tasks:** Includes Fill-Mask, Question Answering, Summarization, Table Question Answering, Text Classification, Text Generation, Text2Text Generation, Token Classification, Translation, Zero-Shot Classification, and Sentence Similarity.
- Libraries:** Includes PyTorch, TensorFlow, and JAX.
- Datasets:** Includes common_voice, wikipedia, squad, bookcorpus, c4, glue, conll2003, and dcep europarl jrc-acquis.
- Languages:** Includes en, es, fr, de, zh, sv, fi, and ru.
- Licenses:** Includes apache-2.0, mit, and cc-by-4.0.
- Other:** Includes AutoTrain Compatible, Infinity Compatible, Eval Results, Carbon Emissions, and Trained with AutoTrain.

A red box highlights the "Tasks" section, with a red arrow pointing to it from a text box that says "Modelos pre-entrenados para resolver diferentes tareas".

A red box highlights the "Datasets" section, with a red arrow pointing to it from a text box that says "Filtros para la búsqueda de modelos".

The main content area shows a list of models. A green box highlights this section, with a green text box that says "Listado de Modelos disponibles". The models listed include:

- distilgpt2
- cross-encoder/ms-marco-MiniLM-L-12-v2
- gpt2
- roberta-base
- bert-base-cased
- bert-base-chinese
- sentence-transformers/all-MiniLM-L6-v2
- sentence-transformers/paraphrase-MiniLM-L6-v2
- bert-base-uncased
- Helsinki-NLP/opus-mt-zh-en
- distilbert-base-uncased
- xlm-roberta-large-finetuned-conll03-english
- distilbert-base-uncased-finetuned-sst-2-english
- distilroberta-base
- facebook/bart-large-mnli
- xlm-roberta-base
- deepset/roberta-base-squad2

<https://huggingface.co/finiteautomata/beto-sentiment-analysis>

 finiteautomata / **beto-sentiment-analysis**  like 7

Text Classification

PyTorch

JAX

Transformers

es

arxiv:2106.09462

bert

sentiment-analysis

Infinity Compatible

Model card

Files and versions

Train

Deploy

Use in Transformers

Sentiment Analysis in Spanish

beto-sentiment-analysis

Repository: <https://github.com/finiteautomata/pysentimiento/>

Model trained with TASS 2020 corpus (around ~5k tweets) of several dialects of Spanish. Base model is BETO, a BERT model trained in Spanish.

Uses POS, NEG, NEU labels.

License

pysentimiento is an open-source library for non-commercial use and scientific research purposes only. Please be aware that models are trained with third-party datasets and are subject to their respective licenses.

1. [TASS Dataset license](#)
2. [SEMEval 2017 Dataset license](#)

Downloads last month
523,852



 **Hosted inference API** 

Text Classification

Mask token: [MASK]

Examples 

Te quiero. Te amo.

Compute

Computation time on cpu: 0.001

NEG

NEU

POS

JSON Output

Maximize

Spaces using finiteautomata/beto-sentiment-analysis

aniket/gradflow-text-classification

cesar/demo-nlp-es

<https://huggingface.co/Helsinki-NLP/opus-mt-es-en>

Hugging Face Search models, datasets, users...

Helsinki-NLP/opus-mt-es-en like 38

Translation Transformers PyTorch TensorFlow Spanish English marian text2text-generation Inference Endpoints License: apache-2.0

Model card Files and versions Community 4

Train Deploy Use in Transformers

Edit model card

spa-eng

- source group: Spanish
- target group: English
- OPUS readme: [spa-eng](#)
- model: transformer
- source language(s): spa
- target language(s): eng
- model: transformer
- pre-processing: normalization + SentencePiece (spm32k,spm32k)
- download original weights: [opus-2020-08-18.zip](#)

Downloads last month
603,116

Inference API

Translation Examples

Los ingredientes de una tortilla de patatas son: huevos, patatas y cebolla

Compute ctrl+Enter

Computation time on Intel Xeon 3rd Gen Scalable cpu: 0.449 s

The ingredients of a potato tortilla are: eggs, potatoes and onion

JSON Output Maximize

Hugging Face – Búsqueda de modelos Open Source

 **Hugging Face**

Models

Datasets

Spaces

Buckets

NEW

Docs

Enterprise

Pricing

Log In

Sign Up

Main

Tasks

Libraries

Languages

Licenses

Other

Tasks

Text Generation

Any-to-Any

Image-Text-to-Text

Image-to-Text

Image-to-Image

Text-to-Image

Text-to-Video

Text-to-Speech

+ 44

Parameters

< 1B

6B

12B

32B

128B

> 500B

Libraries

PyTorch

TensorFlow

JAX

Transformers

Diffusers

GGUF

MLX

MLX

Transformers.js

Safetensors

sentence-transformers

ONNX

+ 42

Apps

vLLM

llama.cpp

MLX LM

LM Studio

Ollama

Jan

Draw Things

+ 8

Inference Providers

Groq

Novita

Cerebras

SambaNova

Nscale

fal

Hyperbolic

Together AI

+ 11

Models

2,908,684

Filter by name

Full-text search

Inference Available

Sort: Most likes

deepseek-ai/DeepSeek-R1

Text Generation • 685B • Updated Mar 27, 2025 • 4.38M • 13.3k

stabilityai/stable-diffusion-xl-base-1.0

Text-to-Image • Updated Oct 30, 2023 • 1.91M • 7.75k

meta-llama/Meta-Llama-3-8B

Text Generation • 8B • Updated Sep 27, 2024 • 2.37M • 6.55k

meta-llama/Llama-3.1-8B-Instruct

Text Generation • 8B • Updated Sep 25, 2024 • 10.6M • 5.89k

bigscience/bloom

Text Generation • 176B • Updated Jul 28, 2023 • 5.56k • 5k

black-forest-labs/FLUX.1-schnell

Text-to-Image • Updated Aug 16, 2024 • 604k • 4.92k

openai/gpt-oss-120b

Text Generation • 120B • Updated Aug 26, 2025 • 5.04M • 4.8k

mistralai/Mixtral-8x7B-Instruct-v0.1

47B • Updated Jul 24, 2025 • 598k • 4.68k

openai/gpt-oss-20b

Text Generation • 22B • Updated Aug 26, 2025 • 8.03M • 4.63k

black-forest-labs/FLUX.1-dev

Text-to-Image • Updated Jun 27, 2025 • 710k • 12.9k

CompVis/stable-diffusion-v1-4

Text-to-Image • Updated Aug 23, 2023 • 618k • 7.01k

hexgrad/Kokoro-82M

Text-to-Speech • Updated Apr 10, 2025 • 11.3M • 6.2k

openai/whisper-large-v3

Automatic Speech Recognition • 2B • Updated Aug 10, 2024 • 5.13M • 0.73k

stabilityai/stable-diffusion-3-medium

Text-to-Image • Updated Aug 12, 2024 • 5.69k • 4.96k

sentence-transformers/all-MiniLM-L6-v2

Sentence Similarity • 22.7M • Updated Mar 6, 2025 • 260M • 4.82k

meta-llama/Llama-2-7b-chat-hf

Text Generation • Updated Apr 17, 2024 • 356k • 4.76k

Tongyi-MAI/Z-Image-Turbo

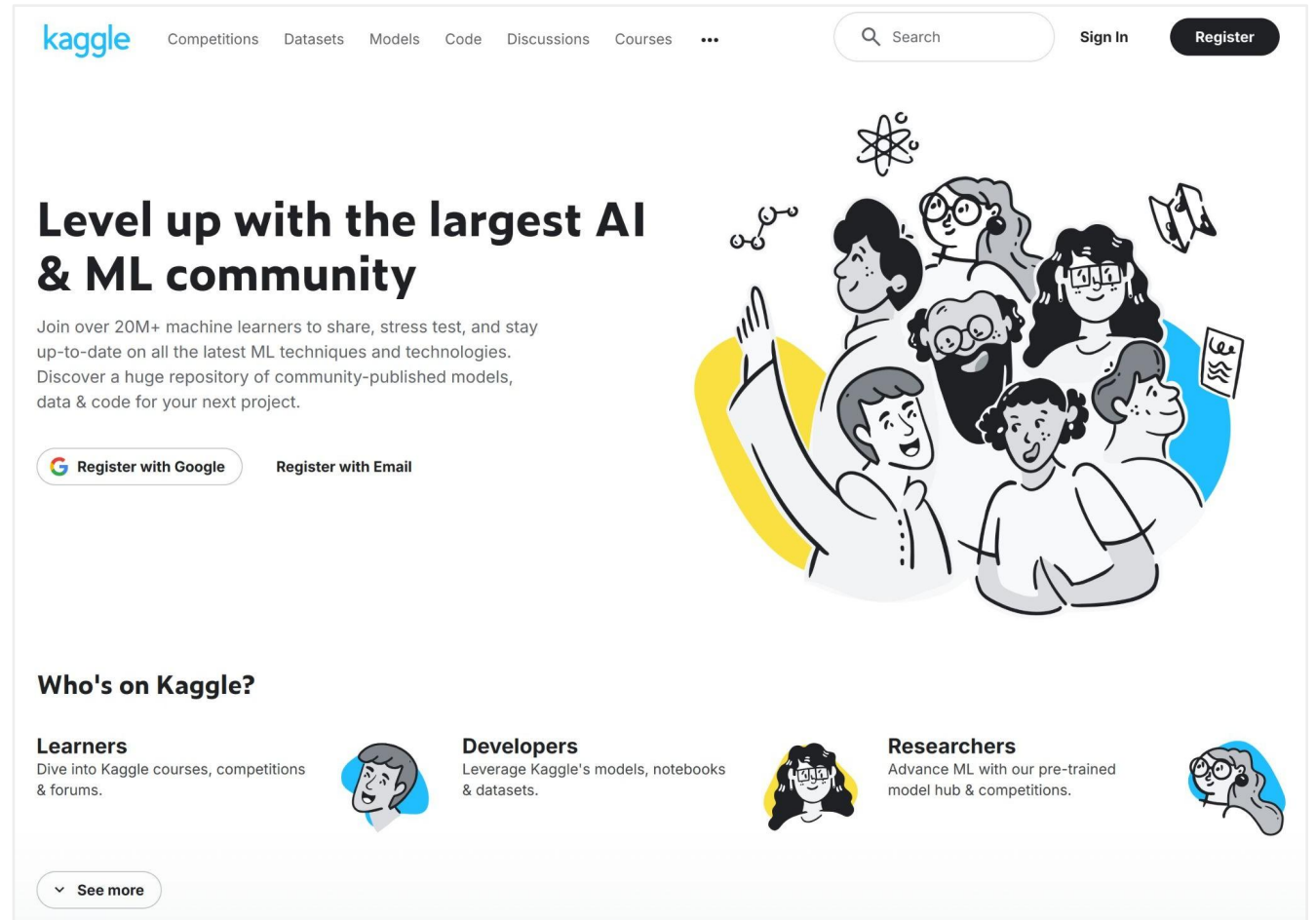
Text-to-Image • Updated Jan 30 • 1.22M • 4.67k

meta-llama/Meta-Llama-3-8B-Instruct

Text Generation • 8B • Updated Jun 19, 2025 • 1.67M • 4.54k

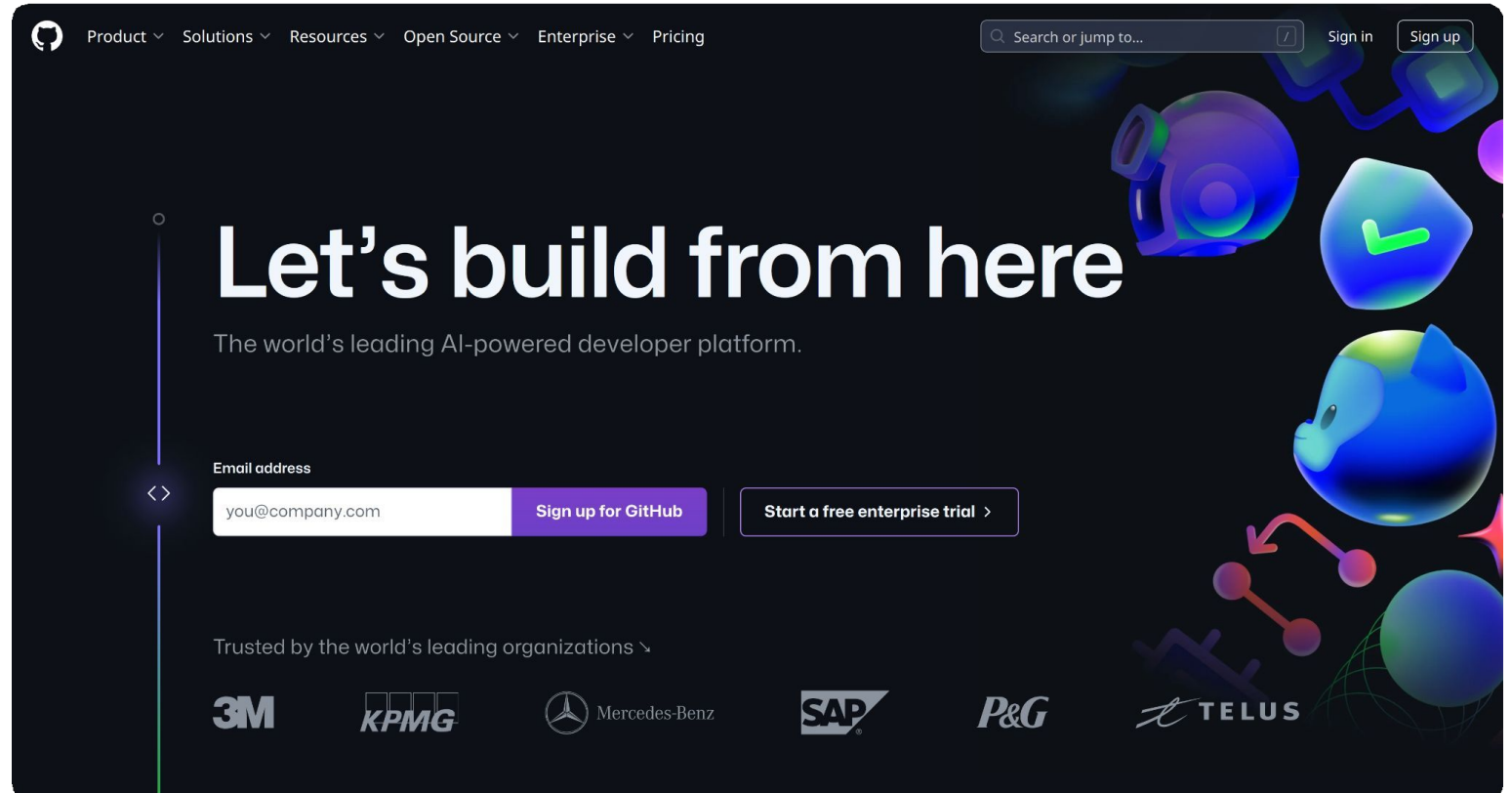
Kaggle

- Comunidad de datos y machine learning
- Datasets públicos para practicar.
- Notebooks ejecutables en la nube.
- Competiciones de ML y data science.
- Ejemplos reales para aprender viendo código de otros.
- URL: <https://www.kaggle.com/>



Github

- Alojar y versionar proyectos.
- Compartir código, documentación y notebooks.
- Colaborar con issues, pull requests y releases.
- Encontrar librerías, ejemplos y proyectos open-source.
- URL: <https://github.com/>



Ejercicios Opcionales



Para quienes quieran practicar con código, este lab recorre la evolución de las principales técnicas de NLP de forma progresiva.

- Bag of Words: representar texto contando palabras
 - Word2Vec: capturar relaciones semánticas entre palabras
 - RNN / LSTM: procesar texto como secuencia
 - Transformers: generar texto con atención multi-cabeza
 - Mini-GPT2: ejemplo práctico de generación de texto
-
- Link al cuaderno Colab:
 - https://github.com/alexoli97/MasterEBIS/blob/main/Lab_NLP.ipynb
 - Link al cuaderno Colab Resuelto:
 - https://github.com/alexoli97/MasterEBIS/blob/main/Lab_NLP_Soluci%C3%B3n.ipynb



Este lab introduce el uso de LLMs desde una API de forma progresiva, explorando cómo cambian las respuestas al modificar los principales parámetros de generación y cómo evaluar resultados de forma básica.

1. Primera llamada a un LLM usando API
2. Uso de mensajes system, user y contexto de conversación
3. Parámetros de generación: temperature, max_tokens, top_p, seed, stop y streaming
4. Prompt engineering básico: rol, objetivo, contexto, formato y restricciones
5. Comparación de configuraciones: respuestas más conservadoras vs más creativas
6. Evaluación manual de respuestas: claridad, exactitud, seguimiento de instrucciones, formato y utilidad
7. Medición básica de coste, tokens y latencia
8. Mini benchmark propio con varios prompts de prueba

Link al cuaderno Colab:

https://github.com/alexoli97/MasterEBIS/blob/main/Lab_LLM_API_Basico_Alumnos.ipynb

Link al cuaderno Colab Resuelto:

https://github.com/alexoli97/MasterEBIS/blob/main/Lab_LLM_API_Basico.ipynb



- En este lab construiremos un asistente capaz de responder preguntas usando documentos como fuente de conocimiento.
- El objetivo es pasar de un LLM que responde solo con su conocimiento interno a un sistema que busca información relevante antes de generar la respuesta.
 - Trabajaremos el flujo básico de un RAG:
 - Carga de documentos.
 - División en chunks.
 - Generación de embeddings.
 - Indexación en una base vectorial.
 - Búsqueda semántica de fragmentos relevantes.
 - Generación de respuesta usando el contexto recuperado.
 - Respuesta con fuentes o referencias.
- El ejercicio permite entender cómo se construyen chatbots sobre PDFs, manuales, documentación interna o bases de conocimiento.
- Link al cuaderno Colab:
- https://github.com/alexoli97/MasterEBIS/blob/main/Lab_RAG_Avanzado_Alumnos.ipynb
- Link al cuaderno Colab Resuelto:
- https://github.com/alexoli97/MasterEBIS/blob/main/Lab_RAG_Avanzado.ipynb



KAHOOT!

Kahoot!



EBIS Business
Techschool

www.ebiseducation.com

